

# Посібник з n8n

## Створено для користувачів n8n будь-якого рівня

Незалежно від того, чи ви лише починаєте свій шлях в автоматизації, чи прагнете покращити вже наявні навички роботи з n8n — цей посібник містить детальні пояснення, практичні приклади та найкращі практики, які допоможуть вам досягти успіху.

## Зміст

1. 📖 Вступ
2. 🚀 Початок роботи з n8n
3. 🔍 Основні поняття
4. 🛠️ Створення вашого першого workflow
5. 🔌 Робота з інтеграціями
6. 🧠 Розширені можливості та функції штучного інтелекту
7. 💡 Практичні приклади та кейси використання
8. 📦 20 готових шаблонів, які можна імпортувати в n8n

---

## 🌟 Розділ 1: Вступ до n8n

### Що таке n8n?

n8n — це потужний і гнучкий інструмент автоматизації робочих процесів, який дозволяє з'єднувати будь-який додаток із API з іншим додатком і керувати його даними з мінімальним або повною відсутністю коду. На відміну від багатьох інших платформ автоматизації, n8n поєднує можливості штучного інтелекту з автоматизацією бізнес-процесів, пропонуючи унікальне поєднання потужності та доступності.

Основою n8n є ліцензійна модель "fair-code" (чесний код), яка надає переваги відкритого програмного забезпечення при збереженні стабільного розвитку. Такий підхід забезпечує користувачам більшу свободу у створенні багатокрокових агентів зі ШІ та інтеграції додатків, ніж традиційні закриті рішення.

n8n вирізняється серед інструментів автоматизації завдяки:

- **Налаштовуваності:** Ви можете створювати робочі процеси з точністю коду або за допомогою інтерфейсу drag-and-drop — найкраще з обох світів.
  - **Гнучкості розгортання:** Обирайте між самостійним хостингом із повним контролем або зручністю хмарного рішення — залежно від ваших потреб.
  - **Орієнтації на конфіденційність:** Варіанти самостійного хостингу дозволяють зберігати дані на вашій власній інфраструктурі для підвищеної безпеки.
  - **Розширюваності:** Можливість створювати власні вузли (nodes) і розширювати функціональність поза межами стандартного функціоналу.
  - **Інтеграції зі ШІ:** Вбудована підтримка створення та реалізації AI-воркфлоу, агентів і інструментів.
- 

## Гнучкість та можливості налаштування

n8n відзначається високою гнучкістю в різних аспектах:

- **Інтеграція з кодом:** Використовуйте вузол *Code* для написання власного JavaScript або Python-коду, коли потрібен точний контроль.
- **Візуальний редактор:** Створюйте робочі процеси за допомогою інтуїтивного drag-and-drop інтерфейсу, коли важлива швидкість.
- **Гібридний підхід:** Поєднуйте візуальну побудову з фрагментами коду для оптимального балансу ефективності й налаштування.
- **Користувацькі вузли:** Розробляйте власні вузли, якщо вбудовані не відповідають вашим потребам.
- **Вирази (Expressions):** Трансформуйте дані без повного кодування — зручно для середнього рівня складності.

Ця гнучкість робить n8n однаково корисним як для технічних користувачів, які потребують глибокого контролю, так і для бізнес-користувачів, яким важливо швидко отримати результат.

---

## Переваги у сфері конфіденційності та контролю даних

У сучасну епоху, коли зростає увага до конфіденційності даних, підхід p8n має низку важливих переваг:

- **Самостійний хостинг:** Зберігайте всі дані та робочі процеси на власних серверах — жодного доступу третіх сторін.
- **Жодного обміну даними:** Ваші дані не передаються стороннім сервісам, якщо ви самі цього не налаштуєте.
- **Підтримка відповідності:** Самохостинг допомагає відповідати вимогам GDPR, HIPAA або галузевим стандартам.
- **Контроль аудиту:** Повні журнали подій та історія змін залишаються у вашій інфраструктурі.
- **Ізоляція мережі:** Можливість запуску воркфлоу в ізольованих мережах без доступу до інтернету.

Ці можливості роблять p8n особливо привабливим для організацій, які працюють із чутливими даними або підпадають під жорстке регулювання.

---

## Відкритий код проти закритих рішень

Модель *fair-code* в p8n — це компроміс між повністю відкритим і закритим (пропрієтарним) ПЗ:

- **Прозорість:** Базовий код відкритий для перегляду, модифікації та навчання.
- **Внески спільноти:** Користувачі можуть додавати нові функції, виправляти помилки та розширювати можливості.
- **Сталий розвиток:** Ліцензія підтримує постійне вдосконалення та підтримку.
- **Комерційне використання:** Чіткі правила ліцензування для бізнес-застосування.
- **Підтримка для підприємств:** Професійна підтримка поєднує гнучкість open-source із надійністю комерційних рішень.

Це дозволяє поєднати переваги відкритого підходу з надійністю та підтримкою закритих платформ.

---

## Хто повинен використовувати p8n?

n8n настільки універсальний, що підходить як окремим користувачам, так і великим корпоративним командам. Ось основні категорії користувачів і те, як вони можуть скористатися n8n:

---

## Технічні команди

### Команди IT-операцій

- **Онбординг/оффбординг співробітників:** Автоматичне створення та налаштування облікових записів у різних системах.
- **Моніторинг систем:** Воркфлоу для контролю стану систем і надсилання сповіщень.
- **Керування тікетами:** Автоматизація розподілу та пріоритезації заявок.
- **Підготовка ресурсів:** Швидке розгортання інфраструктури або сервісів.

### Команди безпеки

- **Реакція на інциденти:** Збагачення тікетів безпеки даними з різних джерел.
- **Збір загроз:** Автоматизація збору й аналізу даних про кіберзагрози.
- **Сканування безпеки:** Регулярне сканування та обробка результатів.
- **Перевірка відповідності:** Автоматизація перевірок і звітності.

### Команди DevOps

- **Інтеграція CI/CD:** Автоматизація процесів у конвеєрі CI/CD.
  - **Перевірка якості коду:** Автоматичні рев'ю та тестування.
  - **Обробка природної мови:** Перетворення команд у звичайній мові в API-запити.
  - **Сповіщення про деплой:** Автоматичні повідомлення про успішне чи невдале розгортання.
- 

## Бізнес-команди

### Відділ продажів

- **Управління лідами:** Обробка й збагачення нових лідів автоматично.

- Аналітика клієнтів: Створення інсайтів з відгуків і відгуків.
- Послідовності фоллоу-апів: Автоматичні дії після взаємодій з клієнтами.
- Звіти з продажу: Генерація та розсилка звітів про продажі.

## Відділ маркетингу

- Автоматизація кампаній: Координація багатоканальних маркетингових активностей.
- Керування соцмережами: Планування та публікація контенту.
- Аналітика маркетингу: Збір і обробка даних.
- Сегментація аудиторії: Автоматичний розподіл користувачів за критеріями.

## Підтримка клієнтів

- Маршрутизація заявок: Розподіл звернень за темами або клієнтами.
  - Шаблони відповідей: Генерація персоналізованих відповідей.
  - Агрегація даних: Отримання повного профілю клієнта з кількох систем.
  - Опитування задоволеності: Відправка та обробка результатів опитувань.
- 

## Індивідуальні користувачі та малі бізнеси

### Автоматизатори-ентузіасти

- Особиста продуктивність: Автоматизація рутинних справ.
- Збір даних: Централізований збір інформації з різних джерел.
- Розумний дім: Інтеграція домашніх пристроїв із зовнішніми сервісами.
- Навчання: Практичне вивчення API та автоматизації.

### Малі бізнеси та стартапи

- Автоматизація процесів: Оптимізація без витрат на розробку.
  - Інтеграція систем: Поєднання окремих інструментів у єдину систему.
  - Масштабування: Почніть з малого й розширюйтесь без зміни платформи.
  - Економія: Автоматизація без дорогого кастомного розроблення.
-

n8n поєднує в собі доступність візуального конструктора й гнучкість програмування, що робить його зручним для широкого кола користувачів — від новачків до інженерів. Якщо вам потрібен контроль, ефективність чи захист даних — n8n стане оптимальним вибором.

Детальніше про можливості n8n ви можете дізнатися на офіційному сайті: [n8n.io](https://n8n.io), де також доступна документація, форум спільноти та інші ресурси для ефективного використання платформи.

---

## Розділ 2: Початок роботи з n8n

### Вибір варіанту розгортання n8n

Одна з головних переваг n8n — це гнучкість у виборі способу розгортання. Ви можете скористатися зручним хмарним рішенням або самостійно хостити платформу для повного контролю над інфраструктурою. У цьому розділі ми розглянемо основні варіанти та допоможемо обрати оптимальний для ваших потреб.

---

### n8n Cloud (хмарне рішення)

n8n Cloud — найшвидший спосіб почати роботу. Це повністю кероване рішення, яке не потребує встановлення, налаштування або обслуговування інфраструктури.

### Основні переваги n8n Cloud:

- **Нуль часу на налаштування:** Можна одразу створювати воркфлоу без встановлення.
- **Автоматичні оновлення:** Завжди доступ до найновіших функцій та оновлень без додаткових дій.
- **Керована інфраструктура:** Вам не потрібно турбуватись про сервери, бази даних чи масштабування.
- **Технічна підтримка:** Доступ до професійної підтримки від команди n8n.
- **Функції для командної роботи:** Інструменти для спільного використання воркфлоу та колаборації.

### Тарифні плани n8n Cloud:

n8n Cloud пропонує кілька планів на вибір:

- **Безкоштовний пробний період:** Спробуйте платформу з обмеженою кількістю запусків і функцій.
- **Personal (особистий):** Для індивідуальних користувачів з помірними потребами.
- **Teams (для команд):** Для малих і середніх команд з потребою у співпраці.
- **Enterprise:** Для великих організацій з вимогами до безпеки, підтримки та масштабованості.

Щоб зареєструватися, перейдіть на сайт [n8n.io](https://n8n.io) і виберіть відповідний план.

---

## Самостійне розгортання (Self-hosted)

Самостійний хостинг надає вам повний контроль над автоматизацією та даними. Ідеально підходить для організацій з особливими вимогами до безпеки, власною інфраструктурою або технічними ресурсами.

## Основні переваги самостійного хостингу:

- **Конфіденційність даних:** Всі дані та облікові записи залишаються у вашій інфраструктурі.
  - **Налаштування:** Можна змінювати та розширювати платформу під власні потреби.
  - **Інтеграція:** Просте підключення до внутрішніх систем, недоступних з хмари.
  - **Контроль витрат:** Можливість знизити витрати при великому обсязі автоматизації.
  - **Відповідність вимогам:** Легше дотримуватися нормативних та галузевих стандартів (GDPR, HIPAA тощо).
- 

## Встановлення через npm

Встановлення через npm (Node Package Manager) — зручний спосіб для користувачів, знайомих із середовищем Node.js.

Необхідні компоненти:

- Node.js (рекомендується версія 16 або новіша)
- npm (входить до складу Node.js)

- База даних (необов'язкова, але рекомендована для продакшну)

## Базові кроки встановлення:

```
npm install n8n -g n8n start
```

Після запуску інтерфейс буде доступний за адресою:

<http://localhost:5678>

---

## Налаштування для продакшн-середовища

Для надійної роботи в продакшн-середовищі слід врахувати:

- Використання менеджера процесів, такого як **PM2**
  - Налаштування окремої бази даних
  - Конфігурація змінних середовища для безпеки
  - Увімкнення автентифікації
- 

## Розгортання через Docker

Docker дозволяє легко розгорнути n8n у контейнеризованому середовищі, забезпечуючи стабільність та повторюваність на різних системах.

Передумови:

- Встановлений Docker на системі
  - **Docker Compose** (рекомендовано для спрощення налаштування)
- 

## Базове налаштування Docker

```
version: '3' services: n8n: image: n8nio/n8n restart: always ports: - "5678:5678" environment: - N8N_BASIC_AUTH_ACTIVE=true - N8N_BASIC_AUTH_USER=user - N8N_BASIC_AUTH_PASSWORD=password volumes: - ~/.n8n:/home/node/.n8n
```

Запустіть n8n за допомогою Docker Compose:



```
docker-compose up -d
```

Інтерфейс n8n буде доступний за адресою: <http://localhost:5678>

## Рекомендації для продакшн-розгортання в Docker

- Використовуйте **secrets** для зберігання конфіденційних даних
- Налаштуйте **постійне зберігання** (persistent storage) для даних воркфлоу
- Забезпечте **правильне мережеве оточення та безпеку**
- Для масштабування розгляньте використання **Docker Swarm** або **Kubernetes**

## Розгортання на сервері

n8n можна встановити на різні серверні платформи. Нижче приклад для Linux (Ubuntu/Debian):

### Linux-сервер (Ubuntu/Debian):

```
# Оновіть систему sudo apt update && sudo apt upgrade -y # Встановіть Node.js
curl -fsSL https://deb.nodesource.com/setup_16.x | sudo -E bash - sudo apt-get install -y nodejs # Встановіть n8n sudo npm install n8n -g # Створіть systemd-сервіс sudo nano /etc/systemd/system/n8n.service
```

Додайте наступний вміст до файлу сервісу:

```
[Unit] Description=n8n After=network.target [Service] Type=simple User=your-user ExecStart=/usr/bin/n8n start Restart=on-failure [Install] WantedBy=multi-user.target
```

Активуйте та запустіть сервіс:

```
sudo systemctl enable n8n sudo systemctl start n8n
```

## Хмарні платформи

n8n також можна розгорнути на основних хмарних провайдерах:

- **AWS:** EC2, ECS або Fargate
- **Google Cloud:** Compute Engine або Google Kubernetes Engine
- **Microsoft Azure:** Azure Container Instances або AKS
- **DigitalOcean:** Droplets або App Platform

Детальні інструкції з налаштування для кожної платформи доступні в офіційній документації n8n.

## Вбудоване використання (Embed) для White-Label-рішень

n8n Embed дозволяє вбудовувати n8n у ваш власний продукт з повним white-label (брендуванням під себе). Це дає змогу пропонувати автоматизацію воркфлоу вашим клієнтам як частину вашого сервісу.

### Основні можливості n8n Embed:

- **White-Label:** Повне налаштування зовнішнього вигляду під ваш бренд
- **Глибока інтеграція:** Вбудована автоматизація прямо у ваш додаток
- **Доступ до API:** Керування n8n через API
- **Кастомна автентифікація:** Інтеграція з вашими системами логіну
- **Пріоритетна підтримка:** Спеціалізована технічна допомога для Embed-кейсів

Використання n8n Embed вимагає індивідуальної угоди з командою n8n. Зв'яжіться з ними через сайт [n8n.io](https://n8n.io) для отримання цін та деталей підтримки.

## Розділ 3: Основні поняття

### Розуміння воркфлоу

У центрі n8n лежить концепція **воркфлоу** — це базовий елемент, що дозволяє створювати автоматизацію. Щоб будувати ефективні рішення для реальних задач, потрібно добре розуміти, як працюють воркфлоу в n8n.

---

## Що таке воркфлоу в n8n?

**Воркфлоу** (workflow) — це набір взаємопов'язаних **вузлів** (nodes), які разом автоматизують певний процес. Уявіть воркфлоу як візуальне представлення послідовних кроків, необхідних для виконання певної задачі.

Кожен воркфлоу складається з:

- **Вузлів (Nodes):** Окремі компоненти, які виконують конкретні дії (наприклад, отримання даних, відправка повідомлення тощо)
- **Зв'язків (Connections):** Лінії між вузлами, які визначають послідовність і потік даних
- **Даних (Data):** Інформація, що передається між вузлами й може трансформуватися в процесі
- **Логіки виконання (Execution Logic):** Правила, які керують тим, як і коли запускається воркфлоу

Воркфлоу в n8n мають такі характеристики:

- **Візуальні:** Легко сприймаються з першого погляду
- **Модульні:** Побудовані з повторно використовуваних блоків
- **Гнучкі:** Можуть адаптуватися до найрізноманітніших сценаріїв
- **Потужні:** Підтримують складні операції й логіку

На відміну від класичного програмування, n8n дозволяє створювати складні автоматизації без написання великої кількості коду. Але за потреби ви завжди можете додати власний скрипт — це дає вам найкраще з обох світів.

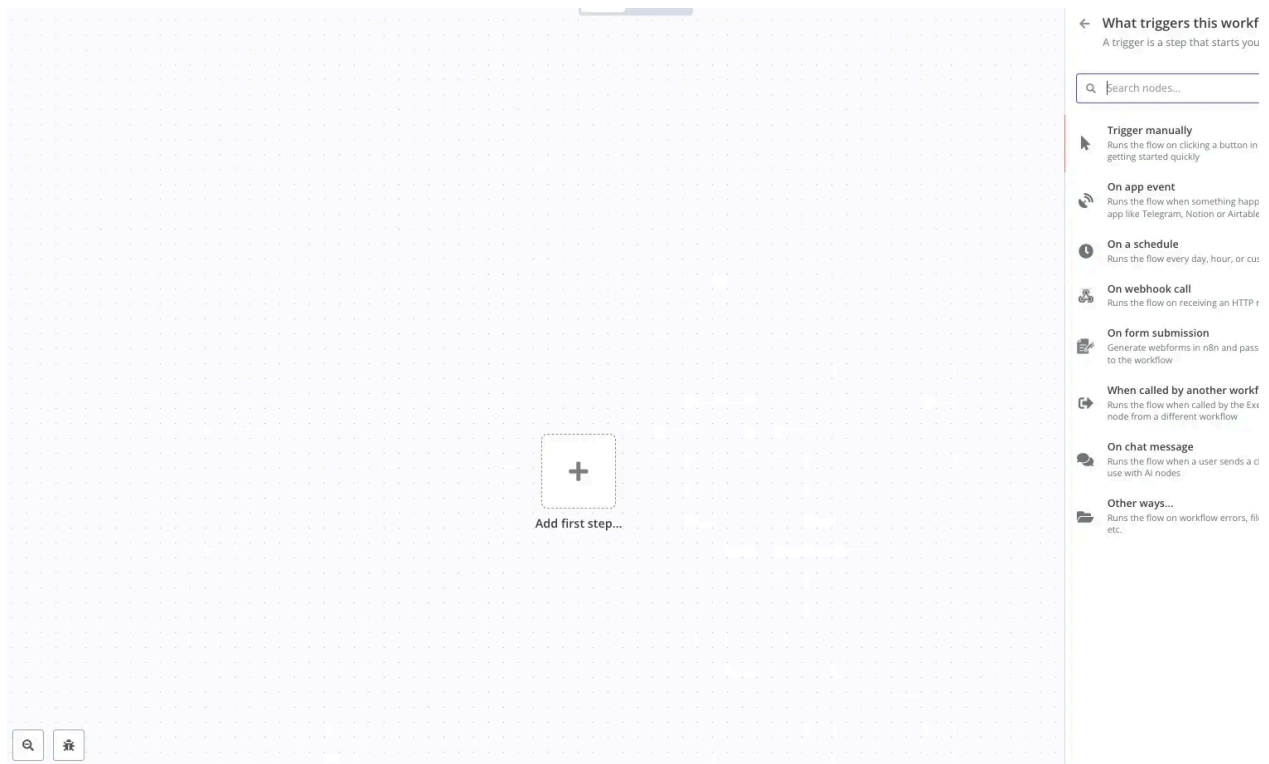
---

## Робоче полотно (Workflow Canvas) та редактор

**Полотно воркфлоу** (workflow canvas) — це ваш робочий простір у n8n. Саме тут ви будуєте автоматизації, додаєте та налаштовуєте вузли, а також з'єднуєте їх між собою.

Інтерфейс побудований таким чином, щоб:

- забезпечити зручне візуальне редагування
- швидко орієнтуватися в логіці процесу
- тестувати та налаштовувати кожен етап вашої автоматизації



## Основні елементи робочого полотна:

- **Область полотна:** Головний простір, де ви розміщуєте та з'єднуєте вузли
- **Панель вузлів:** Пошуковий список доступних вузлів, згрупованих за функціональністю
- **Панель налаштувань вузла:** Опції конфігурації для вибраного вузла
- **Вивід даних виконання:** Показує, як дані проходять через вузли під час тестування
- **Панель інструментів:** Містить дії, такі як збереження, запуск, активація/деактивація

## Робота з полотном:

- **Додавання вузлів:** Знайдіть вузол у панелі й натисніть, щоб додати його на полотно

- **З'єднання вузлів:** Перетягніть з виходу одного вузла до входу іншого
  - **Налаштування вузлів:** Натисніть на вузол, щоб відкрити його конфігурацію праворуч
  - **Переміщення:** Перетягуйте вузли для впорядкування
  - **Зум і панорамування:** Колесо миші — для зуму, пробіл + перетягування — для переміщення по полотну
- 

## Можливості редактора:

- **Автоматичне збереження:** Зміни зберігаються автоматично як чернетка
- **Історія версій:** Можна переглядати зміни та повертатись до попередніх версій
- **Гарячі клавіші:** Прискорюють створення воркфлоу
- **Пошук:** Швидкий пошук вузлів і параметрів
- **Інструменти тестування:** Можливість запуску окремих частин воркфлоу

Інтерфейс редактора інтуїтивно зрозумілий, що дозволяє зосередитись на вирішенні задач, а не боротьбі з інструментом. З досвідом ви зможете створювати дедалі складніші автоматизації з легкістю.

---

## Активні та неактивні воркфлоу

У п8п кожен воркфлоу має два стани: **активний** або **неактивний**. Розуміння різниці між ними — ключ до ефективного керування автоматизацією.

### Неактивні воркфлоу:

- **Стан за замовчуванням:** Усі нові воркфлоу створюються неактивними
- **Лише ручний запуск:** Виконуються тільки при натисканні кнопки "Execute Workflow"
- **Жодних тригерів:** Не реагують на події (вебхуки, розклад тощо)
- **Режим тестування:** Ідеально підходить для розробки та налагодження
- **Не споживають ресурси:** Не навантажують систему, коли не запущені вручну

### Активні воркфлоу:

- **Режим продакшну:** Готові до автоматичної роботи

- **Автоматичне виконання:** Запускаються, коли спрацьовує тригер
  - **Постійне очікування:** Активно слухають події
  - **Використання ресурсів:** Потребують ресурсів системи для підтримки "слухання"
  - **Справжня автоматизація:** Повністю автономне виконання процесів
- 

## Коли використовувати кожен стан:

Використовуйте *неактивний* стан під час:

- Початкової розробки
- Тестування й відлагодження
- Внесення значних змін
- Тимчасового вимкнення воркфлоу

Використовуйте *активний* стан, коли:

- Воркфлоу готовий до використання
  - Потрібна автоматична реакція на події
  - Воркфлоу є частиною критичних бізнес-процесів
  - Ви хочете "налаштувати і забути" автоматизацію
- 

## Активація та деактивація воркфлоу

Щоб перемикатись між станами:

1. Відкрийте воркфлоу в редакторі
2. Натисніть перемикач "**Active/Inactive**" у верхньому правому куті
3. Підтвердіть вибір, якщо буде запропоновано

⚠ Пам'ятайте: якщо у воркфлоу є тригер (наприклад, Webhook або Schedule), після активації він одразу почне реагувати на події. Завжди тестуйте перед активацією в продакшні.

---

## Режими виконання воркфлоу

n8n підтримує різні режими виконання, що дозволяє налаштувати систему під потреби автоматизації та доступні ресурси.

---

## Звичайний режим виконання (Regular Execution Mode)

Це режим за замовчуванням:

- **Процес:** Виконується у головному процесі n8n
  - **Пам'ять:** Спільне використання пам'яті з n8n
  - **Найкраще для:** Простих воркфлоу та тестування
  - **Обмеження:** Може вплинути на продуктивність, якщо воркфлоу важкий
- 

## Режим черги (Queue Mode)

Розділяє виконання з основним процесом:

- **Процес:** Завдання ставляться в чергу та виконуються по черзі
  - **Пам'ять:** Краща ізоляція між завданнями
  - **Найкраще для:** Продакшн із великою кількістю воркфлоу
  - **Переваги:** Стабільніша продуктивність і передбачуване використання ресурсів
  - **Налаштування:** Вмикається через змінні середовища
- 

## Режим виконання Webhook (Webhook Execution Mode)

Призначений для воркфлоу з тригерами Webhook:

- **Процес:** Виконується одразу після отримання Webhook
- **Відповідь:** Повертає відповідь прямо викликаючій стороні
- **Найкраще для:** API-подібних сценаріїв, де важлива миттєва реакція
- **Приклад використання:** Створення кастомних API-ендпоінтів за допомогою n8n

## Ручне проти автоматичного виконання

Воркфлоу в n8n можна запускати двома способами:

## Налаштування поведінки виконання

n8n дозволяє налаштовувати параметри поведінки виконання воркфлоу:

- **Timeout** (*тайм-аут*) — максимальний час виконання воркфлоу
- **Retry On Failure** — автоматичне повторне виконання у разі помилки
- **Error Workflow** — запуск іншого воркфлоу у разі помилки
- **Success Workflow** — запуск іншого воркфлоу після успішного виконання
- **Execution Data Saving** — контроль обсягу збережених даних виконання

Ці параметри можна задавати як **глобально** (для всіх воркфлоу), так і **індивідуально** — для кожного окремого воркфлоу. Це дозволяє точно налаштовувати поведінку автоматизації відповідно до потреб.

---

## Вузли: будівельні блоки

**Вузли (Nodes)** — це основні елементи воркфлоу в n8n. Кожен вузол виконує окрему дію, ініціює запуск або обробляє дані. У сукупності вони формують повний автоматизований процес.

---

## Типи вузлів та категорії

n8n класифікує вузли за типами та категоріями, щоб полегшити пошук потрібного інструменту під час створення воркфлоу.

### Основні типи вузлів:

- **Тригер-вузли (Trigger Nodes)**

Запускають воркфлоу у відповідь на подію

- **Webhook**: Спрацьовує при HTTP-запиті
- **Schedule**: Виконується у заданий час
- **Event-based**: Реагує на події у зовнішніх системах

- **Звичайні вузли (Regular Nodes)**

Виконують дії після запуску воркфлоу

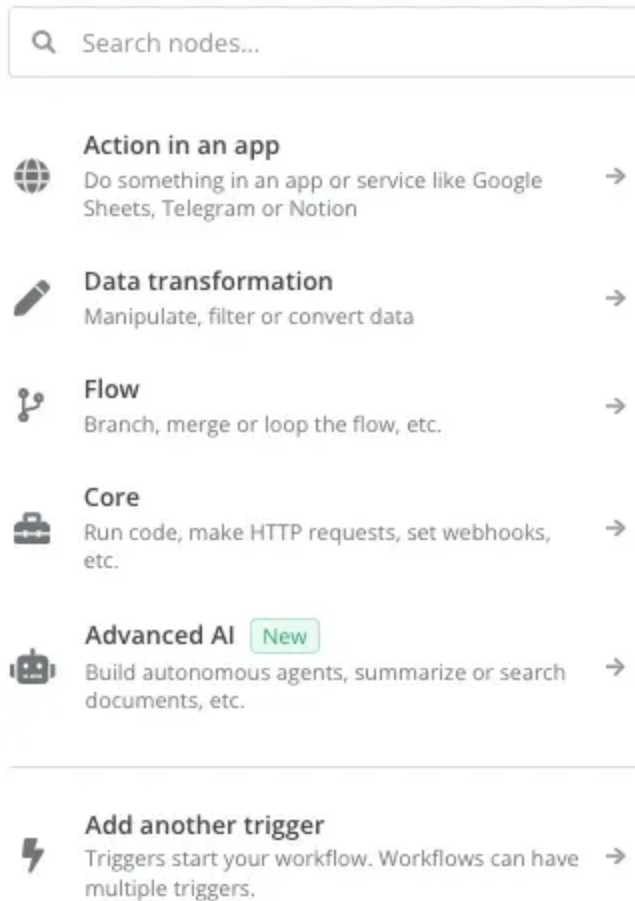
- **Action**: Виконання операцій у зовнішніх сервісах
- **Helper**: Обробка, зміна або трансформація даних
- **Flow**: Контроль логіки виконання (розгалуження, переходи тощо)



- **Базові вузли (Core Nodes)**

Вбудовані функції для поширених задач

- **HTTP Request:** Відправка API-запитів
- **Function:** Виконання JavaScript-коду
- **IF/Switch:** Реалізація умовної логіки
- **Merge:** Об'єднання даних з кількох джерел



## **Функціональні категорії вузлів в n8n**

Вузли в n8n згруповані за сферою застосування, що дозволяє швидко знайти потрібні інструменти автоматизації.

### **Комунікації**

- **Email:** Gmail, SMTP

- **Месенджери:** Slack, Discord, Telegram
- **SMS:** Twilio, MessageBird

## CRM та Продажі

- **CRM-системи:** HubSpot, Salesforce, Pipedrive
- **Призначення:** управління клієнтами та автоматизація продажів

## Обробка даних

- **Таблиці:** Google Sheets, Excel
- **Бази даних:** MySQL, PostgreSQL, MongoDB
- **Призначення:** трансформація та аналіз даних

## Розробка

- **Git-провайдери:** GitHub, GitLab
- **Трекінг задач:** Jira, Linear
- **CI/CD:** Jenkins, CircleCI

## Маркетинг

- **Соцмережі:** Twitter, Facebook, LinkedIn
- **Email-маркетинг:** Mailchimp, SendGrid
- **Аналітика:** відстеження кампаній, поведінки користувачів

## Продуктивність

- **Управління проєктами:** Asana, Trello
- **Нотатки:** Notion, Evernote
- **Файлові системи:** Dropbox, Google Drive

## Штучний інтелект і машинне навчання

- **Провайдери ШІ:** OpenAI, LangChain
- **Можливості:** генерація тексту й зображень, аналіз даних, прогнозування

Кожен вузол виконує конкретну функцію — від базових маніпуляцій з даними до інтеграцій з хмарними сервісами. Комбінуючи вузли, можна автоматизувати практично будь-який бізнес-процес.

---

## Тригери vs. Звичайні вузли

### Тригери (Trigger Nodes)

Тригер-вузли — це **стартові точки воркфлоу**, які слідкують за подіями та **автоматично** запускають воркфлоу при їх настанні.

#### Основні характеристики:

- Розміщуються на **початку воркфлоу**
- **Тільки один** активний тригер у воркфлоу
- Необхідні для **автоматичного виконання**
- Визначають, **коли і чому** запускається воркфлоу

#### Типові тригери:

- **Webhook Trigger**: запускається при HTTP-запиті  
*Наприклад: надсилання форми або API-запит*
- **Schedule Trigger**: виконується за розкладом  
*Наприклад: щоденні звіти або автоматичні резервні копії*
- **Polling Trigger**: перевіряє наявність змін з інтервалом  
*Наприклад: вхідна пошта, нові рядки в таблиці*
- **Event-Based Trigger**: реагує на події у зовнішніх системах  
*Наприклад: нове повідомлення в Slack, коміт у GitHub*

---

### Звичайні вузли (Regular Nodes)

Ці вузли **виконують дії** після запуску воркфлоу. Вони можуть:

- Обробляти й трансформувати дані
- Виконувати API-запити

- Реалізовувати логіку контролю потоку

## Основні характеристики:

- Розміщуються після тригерів
- Безлімітна кількість у воркфлоу
- Виконуються **послідовно** згідно з логікою зв'язків
- Реалізують основну роботу автоматизації

## Типи звичайних вузлів:

- **Action Nodes:** взаємодія із зовнішніми сервісами  
*Наприклад: надсилання листа, створення запису в базі*
- **Helper Nodes:** обробка та трансформація даних  
*Наприклад: форматування тексту, фільтрація масивів*
- **Flow Control Nodes:** управління маршрутом виконання  
*Наприклад: умовні блоки IF, перемикачі Switch*
- **Core Utility Nodes:** базові дії  
*Наприклад: HTTP-запити, виконання JavaScript*



## Основні відмінності між тригерами і звичайними вузлами

| Аспект             | Тригер-вузли              | Звичайні вузли                          |
|--------------------|---------------------------|-----------------------------------------|
| Позиція у воркфлоу | Завжди на початку         | Після тригера                           |
| Призначення        | Ініціюють виконання       | Виконують конкретні дії                 |
| Кількість          | Один активний на воркфлоу | Необмежена кількість                    |
| Запуск             | Чекають на зовнішні події | Виконуються, коли до них доходить потік |
| Стан воркфлоу      | Потрібен активний стан    | Працюють в будь-якому стані             |

## Розуміння різниці між типами вузлів

Розуміння відмінностей між тригер-вузлами та звичайними вузлами допомагає створювати **коректні, логічні та ефективні воркфлоу**, які належним чином реагують на події та виконують дії в правильному порядку.

---

## Налаштування та конфігурація вузлів

Кожен вузол у n8n можна налаштувати так, щоб він виконував певну дію відповідно до ваших потреб. Грамотне налаштування вузлів є критично важливим для створення надійних автоматизацій.

---

### Базова конфігурація вузла

#### Додавання вузла:

1. Знайдіть потрібний вузол у панелі вузлів (зліва).
2. Натисніть на нього — він з'явиться на полотні.
3. Перемістіть його в потрібну позицію у вашому воркфлоу.

#### Відкриття налаштувань вузла:

- Клікніть на вузол на полотні.
  - Справа з'явиться панель конфігурації (Settings Panel).
- 

### Основні параметри налаштувань:

- **Node Name (Ім'я вузла):**

Дайте вузлу зрозумілу назву (наприклад, "Отримати дані клієнта", а не "HTTP Request").

- **Connection (З'єднання):**

Виберіть або створіть облікові дані (credentials), якщо це необхідно для інтеграції з API.

- **Operation (Операція):**

Виберіть дію, яку має виконати вузол (наприклад, "GET", "SEND", "CREATE" тощо).

- **Parameters (Параметри):**

Задайте додаткові налаштування для цієї дії (адреса URL, фільтри, текст повідомлення тощо).

---

## Розширені параметри

### Вирази (Expressions):

- Дозволяють використовувати **динамічні значення** із попередніх вузлів.
- Можна доступатись до полів попередніх вузлів, будувати умовні вирази.
- Підходить для **форматування, розгалуження, автоматичної генерації значень**.

### Обробка помилок (Error Handling):

- Налаштування повторних спроб при помилках (retries).
- Встановлення інтервалів між спробами.
- Задання альтернативних значень або дій при помилці.

### Обробка сторінок (Pagination):

- Якщо API повертає **порційні результати (пагінацію)**:
    - Можна налаштувати кількість об'єктів на сторінку.
    - Визначити, як переходити між сторінками.
    - Обмежити загальну кількість оброблюваних елементів.
- 

## Рекомендації з налаштування вузлів

- **Використовуйте зрозумілі назви вузлів**

*Приклад: "Get Customer Data" замість "HTTP Request"*

- **Тестуйте поступово**

Використовуйте кнопку **"Execute Node"**, щоб перевірити кожен крок окремо.

- **Документуйте складні налаштування**

- Додавайте нотатки до вузлів.
- Коментуйте код у функціональних вузлах (Function, Code).

- Оптимізуйте продуктивність
  - Забирайте тільки необхідні дані.
  - Використовуйте фільтри на джерелі.
  - Уникайте зайвих обчислень при великому обсязі даних.

Добре налаштовані вузли = стабільний, зрозумілий і продуктивний воркфлоу.

## Структура вхідних та вихідних даних

Щоб ефективно працювати з n8n, потрібно розуміти, як дані проходять через вузли. Кожен вузол отримує вхідні дані, обробляє їх і передає вихідні дані наступному вузлу.

### Базова структура даних

n8n використовує єдину узгоджену JSON-структуру:

### Масив елементів (*Items Array*)

- Усі дані організовані у вигляді масиву об'єктів
- Кожен об'єкт (item) — це пара “ключ-значення”
- Кожен item подібний до рядка в таблиці або одного запису

### Приклад структури item'ів:

```
[ { "key1": "value1", "key2": "value2", "nested": { "subkey": "subvalue" } },  
  { "key1": "another value", "key2": "yet another value" } ]
```

- Ключі — це назви полів
- Значення — це дані, які передаються далі
- Вкладені об'єкти допускаються

Важливо: кожен вузол отримує масив об'єктів, обробляє кожен елемент по черзі, а потім передає результат далі у тому ж форматі.



## Бінарні дані (Binary Data)

- Бінарні дані (наприклад, файли, зображення) зберігаються у спеціальній властивості `binary`.
  - Сам вміст файлів зберігається **n8n** внутрішньо й доступний за унікальним ключем.
  - Приклад використання: передавання файлів між вузлами або надсилання вкладень у листах.
- 



## Потік даних між вузлами



### Вхідні дані (Input Data)

- Перший вузол (зазвичай тригер) **генерує початкові дані**.
- Наступні вузли отримують дані від **попередніх**.
- За допомогою **виразів (expressions)** можна звертатися до **даних з будь-якого попереднього вузла**.



### Обробка даних (Data Processing)

- Кожен вузол **обробляє вхідні дані** згідно зі своєю конфігурацією.
- Типові трансформації:
  - **Фільтрація** (відсів непотрібного)
  - **Мапінг** (зміна структури)
  - **Збагачення** (додавання нових полів)
  - **Перетворення формату**



### Вихідні дані (Output Data)

- Кожен вузол **створює результат** після обробки.
  - Цей результат стає **вхідними даними** для наступного вузла.
  - Результат **останнього вузла** — це результат усього воркфлоу.
- 



## Шаблони трансформації даних



## One-to-One (Один до одного)

- Кожен вхідний елемент → один вихідний
- Кількість елементів не змінюється
- *Приклад:* додавання поля до кожного запису клієнта

## Фільтрація (Filtering)

- Деякі елементи не потрапляють у результат
- Вихідних елементів менше, ніж вхідних
- *Приклад:* залишити лише клієнтів з певного регіону

## Розгортання (Expansion)

- Один вхідний елемент → кілька вихідних
- *Приклад:* розділити список зі значеннями через кому на окремі рядки

## Агрегація (Aggregation)

- Об'єднання кількох вхідних елементів у один або кілька
- *Приклад:* групування продажів по регіону й підрахунок підсумків

## Об'єднання (Merging)

- Дані з кількох шляхів з'єднуються
- *Приклад:* злиття інформації про клієнта та історії його замовлень

---

## Перегляд і налагодження даних

n8n надає вбудовані інструменти для аналізу та відстеження даних на кожному етапі воркфлоу:

### Панель виконання (Execution Data Display)

- Показує вхідні та вихідні дані для кожного вузла
- Доступна під час і після запуску воркфлоу
- Допомогає виявити проблеми в трансформації даних

## JSON View

- Повний вигляд даних у форматі JSON
- Можна згортати/розгортати вкладені об'єкти
- Є пошук по ключах

## Table View

- Показ даних у табличному форматі
- Зручно для масивів подібних об'єктів (як у таблицях)
- Автоматично "розплющує" перший рівень вкладених даних

## Binary Data View

- Показує інформацію про бінарні файли
- Включає назву файлу, MIME-тип, розмір
- Дає попередній перегляд, якщо формат підтримується

Добре налагоджений потік даних — ключ до надійної автоматизації.

---

## Потік даних та обробка

Ефективна робота з даними — основа будь-якої успішної автоматизації. В n8n потрібно розуміти не лише структуру, а й як саме дані передаються між вузлами.

---

## Принципи потоку даних

- **Послідовність (Sequential Processing)**
  - Дані зазвичай передаються зліва направо
  - Вузли виконуються у тому порядку, як з'єднані
  - Результат одного вузла — вхід для наступного
- **Типи з'єднань**
  - **Стандартне:** передає всі елементи далі
  - **Основний/альтернативний вихід:** використовується в умовних вузлах (IF, Switch)

- **Гілки (Multiple Paths)**

- Воркфлоу можуть мати **розгалуження**
  - Кожна гілка може обробляти дані незалежно
  - Гілки можуть **знову з'єднуватися**
- 



## **Механізми передавання даних**

- **Пряме передавання (Direct Passing)**

- Найбільш поширений варіант
- Повний масив item'ів передається далі
- Структура масиву зберігається

- **Вибіркове передавання (Selective Passing)**

- Деякі вузли можуть передавати **частину** елементів
  - *Приклад:* вузол **IF** передає елементи в залежності від умови ("true" або "false")
- 



## **Передача за посиланням (Reference Passing)**

- Вузли можуть звертатися до **даних з будь-якого попереднього вузла**, використовуючи **вирази (expressions)**
  - Не обмежується тільки безпосереднім попереднім вузлом
  - Дозволяє реалізовувати **складні логіки об'єднання та збагачення даних**
- 



## **Порядок виконання**



### **Послідовність виконання вузлів**

- Вузли виконуються в **порядку, визначеному їх зв'язками**
- Вузол виконується **тільки після завершення всіх його вхідних вузлів**
- Розгалуження виконуються **паралельно**, якщо вони не залежать одне від одного

## Обробка елементів (Item Processing)

- За замовчуванням кожен вузол обробляє **всі елементи масиву item'ів**
- Деякі вузли можна налаштувати для **індивідуальної обробки**
- Порядок елементів **зберігається**, якщо ви явно його не змінюєте

## Візуалізація виконання

- n8n **підсвічує вузли** під час виконання
- Завершені вузли мають **індикатори статусу**: успішно/помилка
- Ви можете **переглядати дані** на кожному етапі — під час або після виконання

Розуміння порядку виконання та передачі даних — ключ до збереження послідовності і цілісності даних у складних автоматизованих процесах.

## Основи трансформації даних

Дані **рідко бувають готові "з коробки"**. Часто необхідно їх змінювати або приводити до потрібного формату. n8n надає потужні інструменти для цього.

## Типові потреби в трансформації

### Зміна структури:

- Перейменування полів
- Перетворення вкладених об'єктів
- Конвертація форматів (наприклад, JSON → текст)
- Сплющення або вкладення структури

### Зміна значень:

- Форматування тексту чи чисел
- Перетворення дат/часу
- Арифметичні розрахунки
- Операції над рядками (об'єднання, розділення тощо)

## Операції над колекціями:

- Фільтрація масивів
  - Сортування елементів
  - Групування за критеріями
  - Агрегація значень (наприклад, підсумки, середнє тощо)
- 

## Методи трансформації в n8n

- **Set Node**: базовий інструмент для мапінгу й створення нових полів
  - **Function Node**: складна логіка на JavaScript
  - **Item Lists Node**: для роботи з масивами (розділення, об'єднання, фільтрація)
  - **Spreadsheet File Node**: трансформації табличних даних (CSV, Excel)
- 

## Приклади трансформацій

### Просте мапінг-перетворення (Set Node)

```
// Вхід: { "firstName": "John", "lastName": "Doe", "email": "john@example.com" } // Вихід: { "name": "John Doe", "contact": "john@example.com" }
```

### Умовна трансформація (вирази)

```
// Вхід: { "status": "active", "daysRemaining": 5 } // Вихід: { "statusDisplay": "Active (5 days remaining)", "alertLevel": "normal" }
```

## Рекомендації з трансформації даних

- Трансформуйте рано

Приводьте дані до потрібного формату на початку воркфлоу

- Використовуйте правильний інструмент
  - Просте перетворення → **Set Node**
  - Складна логіка → **Function Node**
  - Масиви → **Item Lists Node**
- Зберігайте цілісність даних
  - Валідуйте значення перед змінами
  - Обробляйте порожні або null-поля
  - Зберігайте "сирі" (оригінальні) дані для відстеження, якщо потрібно

Володіння трансформацією даних у n8n дозволяє реалізовувати будь-які сценарії автоматизації, незалежно від вихідного формату чи структури джерела.

## ! Обробка помилок у воркфлоу

Надійна автоматизація неможлива без **грамотної обробки помилок**. У n8n є вбудовані механізми для виявлення, керування та реагування на непередбачувані ситуації, щоб воркфлоу працювали стабільно.

## Типи помилок у n8n

### Помилки виконання вузлів:

- Невдалий API-запит
- Неправильний формат вхідних даних
- Помилка автентифікації (логін/пароль/токен)
- Перевищено час очікування (timeout)

### Помилки обробки даних:

- Відсутні обов'язкові поля
- Невідповідність типів (наприклад, число замість рядка)
- Невалідні розрахунки

- Звернення до неіснуючого індексу масиву

## ✖ Помилки дизайну воркфлоу:

- Неправильне з'єднання вузлів
  - Нескінченні цикли
  - Відсутні облікові дані
  - Логічні помилки (неправильні умови, порядок виконання тощо)
- 

## 🔧 Вбудовані засоби обробки помилок

### 1. Error Workflows

- Окремий воркфлоу, який запускається при помилці в основному
  - Отримує інформацію про помилку та контекст
  - Може виконати дії відновлення, надіслати повідомлення, зберегти лог
- 

### 2. Continue on Fail

- Установка на рівні вузла
  - Дозволяє продовжити виконання, навіть якщо вузол завершився з помилкою
  - Корисно для некритичних дій (наприклад, логування, сповіщення)
  - Доступне у більшості вузлів в опціях налаштування
- 

### 3. Error Trigger Node

- Запускає окремий воркфлоу, якщо інший воркфлоу завершився з помилкою
  - Отримує повну інформацію про виняток
  - Можна фільтрувати за конкретним воркфлоу або типом помилки
- 

### 4. Try/Catch логіка (імітація)

- Вбудованої конструкції `try/catch` у n8n немає

- Але її можна моделювати через **Function Node** — логіка з перехопленням помилок

### Приклад:

```
try { const result = JSON.parse($item.potentiallyInvalidJson); return { json: { success: true, data: result } }; } catch (error) { return { json: { success: false, error: error.message } }; }
```

## Умовні шляхи в залежності від результату

Використовуйте **IF Node** для перевірки, чи вдалася операція, й направлення потоку у відповідну гілку.

### Умовна перевірка:

```
{{ $item.success === true }}
```

## Значення за замовчуванням для відсутніх даних

### У вузлі Set Node або виразах:

```
{ "name": $item.name || "Невідомо", "price": $item.price !== undefined ? $item.price : 0, "category": $item.category || "Без категорії" }
```

## ✓ Перевірка даних перед критичними діями

### У Function Node:



```
const item = $input.item.json; const errors = []; if (!item.email) errors.push("Email обов'язковий"); if (!item.name) errors.push("Ім'я обов'язкове"); if (item.amount <= 0) errors.push("Сума має бути більшою за нуль"); if (errors.length > 0) { return { json: { valid: false, errors } }; } else { return { json: { ...item, valid: true } }; }
```

## Розширені методи: Механізми повтору (Retry)

Приклад з експоненційною затримкою:

```
async function makeRequestWithRetry(url, maxRetries = 3, delay = 1000) { let lastError; for (let attempt = 0; attempt < maxRetries; attempt++) { try { const response = await $http.request({ method: 'GET', url: url, returnFullResponse: true }); return response; } catch (error) { lastError = error; await new Promise(resolve => setTimeout(resolve, delay)); delay *= 2; // експоненційна затримка } } throw new Error(`Не вдалося після ${maxRetries} спроб. Помилка: ${lastError.message}`); } try { const response = await makeRequestWithRetry($item.url); return { json: { success: true, data: response.data } }; } catch (error) { return { json: { success: false, error: error.message } }; }
```

## Поради з обробки помилок

- Завжди тестуйте вузли на невалідні вхідні дані
- Використовуйте **Continue on Fail** для обробки непередбачених збоїв
- Розділяйте критичні та допоміжні процеси
- Використовуйте **Error Workflow** для централізованого логування
- Валідовуйте дані перед надсиланням у зовнішні системи
- Створюйте fallback-шляхи через IF + Set Node
- Документуйте логіку обробки помилок у вузлах (через Node Notes)

## **Логування помилок і сповіщення**

- Створюйте окремі воркфлоу для логування помилок
  - Відправляйте сповіщення про критичні збої (на email, Slack, Telegram тощо)
  - Зберігайте деталі помилок у базу даних або файли для подальшого аналізу
- 

## **Гнучке деградування (Graceful Degradation)**

- Дизайнуйте воркфлоу так, щоб при відмові не критичних вузлів процес продовжувався
  - Реалізуйте альтернативні джерела даних або обхідні дії
- 

## **Транзакційні шаблони**

- Для критичних дій реалізуйте відкат (rollback) змін у разі помилок
  - Зберігайте початковий стан перед внесенням змін
  - Якщо процес не вдається повністю — відновлюйте початкові дані
- 

## **Найкращі практики обробки помилок**

- Передбачайте ймовірні точки збою
  - Додавайте обробку саме там, де найбільше шансів на помилку
- Тестуйте сценарії з помилками
  - Навмисно викликайте помилки для перевірки поведінки воркфлоу
- Надавайте змістовні повідомлення про помилки
  - Додавайте контекст: який вузол, які дані, чому сталася помилка
- Аналізуйте й вчіться
  - Відстежуйте повторювані помилки
  - Вдосконалюйте валідацію й логіку на основі історії збоїв

- **Документуйте обробку помилок**
  - Заносьте логіку обробки помилок у опис воркфлоу або нотатки до вузлів
  - Додавайте інструкції для усунення несправностей

 Ефективна обробка помилок = надійна автоматизація без ручного втручання.

---

## Розділ 4: Створення вашого першого воркфлоу

---

### Основи створення воркфлоу

Процес створення воркфлоу в n8n досить простий після розуміння базових принципів. У цьому розділі ви пройдете через усі ключові кроки — від налаштування до запуску.

---

### Створення нового воркфлоу

#### З головної панелі:

- **Перейдіть у розділ Workflows**
  - Відкрийте головну панель n8n
  - У лівому меню натисніть "Workflows"
- **Створіть новий воркфлоу**
  - Угорі праворуч натисніть кнопку "+ Create"
  - Оберіть "Workflow" зі списку
  - (Опціонально) виберіть, чи створити його в особистому просторі або в проєкті

- **Назвіть воркфлоу**
    - Натисніть на назву "My workflow" у верхній частині редактора
    - Введіть змістовну назву, яка відображає призначення
    - (Опціонально) додайте теги для класифікації
  - **Додайте опис** (необов'язково, але рекомендується)
    - Клацніть на значок інформації "i" біля назви
    - Опишіть призначення воркфлоу, вимоги, важливі деталі
- 

## Орієнтація на полотні воркфлоу

Після створення нового воркфлоу ви побачите **полотно**, де будується логіка автоматизації.

- **Центральне полотно** – основний простір для розміщення вузлів
  - **Панель вузлів** – зліва: усі доступні вузли, згруповані за функціями
  - **Панель налаштувань** – справа: параметри вибраного вузла
  - **Панель інструментів** – зверху: збереження, запуск, активація
- 

## Додавання та з'єднання вузлів

Вузли — це **блоки**, з яких будується воркфлоу. Їхнє правильне додавання та з'єднання — ключ до успішної автоматизації.

---

### + Додавання вузлів

- **Додайте перший вузол**
  - Натисніть "Add first step..." у центрі полотна
  - Або:
    - Клікніть "+" при наведенні на крапку з'єднання
    - Знайдіть вузол у лівій панелі та натисніть на нього
    - Перетягніть вузол на полотно мишкою

- **Пошук вузлів**
    - Використовуйте **пошуковий рядок** у верхній частині панелі вузлів
    - Вводьте назви сервісів або дій (наприклад, "HTTP", "Slack", "Email")
    - Переглядайте категорії для натхнення
  - **Розміщення вузлів**
    - Впорядковуйте вузли **зліва направо** (логіка виконання)
    - Залишайте простір між ними для зручності
    - Групуйте вузли за функцією (наприклад, API → обробка → збереження)
- 

## З'єднання вузлів

- **Базове з'єднання**
    - Наведіть курсор на **правий бік вузла** (вихідна крапка)
    - Потягніть лінію до **лівого боку іншого вузла** (вхідна крапка)
    - Відпустіть — з'єднання створено
  - **Типи з'єднань**
    - **Стандартне з'єднання**: суцільна лінія, передає всі дані
    - **Умове з'єднання**: використовується з вузлами IF або Switch для поділу потоку
- 

## Керування з'єднаннями між вузлами

### Дії зі з'єднаннями:

- **Видалити з'єднання**
  - Клікніть на лінію з'єднання та натисніть клавішу **Delete**
  - Або: натисніть праву кнопку миші → **"Remove"**
- **Змінити маршрут з'єднання**
  - Видаліть поточне з'єднання
  - Створіть нове, тягнучи лінію до іншого вузла

- Перегляд потоку даних
  - Під час виконання n8n показує **кількість елементів**, що проходять через кожне з'єднання

### Рекомендації щодо з'єднань:

- Уникайте перехрещення ліній — це ускладнює читання
- Використовуйте функцію "Auto-arrange"  
(Правий клік на полотні → "Auto-arrange")
- Пам'ятайте: дані течуть у напрямку стрілки з одного вузла до іншого

---

## Налаштування параметрів вузла

Кожен вузол у n8n потребує налаштувань відповідно до обраної дії. **Правильна конфігурація** — запорука успішного виконання воркфлоу.

---

### Базові кроки налаштування вузла

1. **Виберіть вузол**
  - Клікніть на вузол, щоб відкрити панель налаштувань праворуч
2. **Оберіть операцію**
  - Багато вузлів мають кілька дій (наприклад: Create, Update, Delete)
  - Виберіть потрібну з випадаючого списку
3. **Заповніть обов'язкові параметри**
  - Вони позначені зірочкою
  - Введіть потрібні значення або динамічні вирази
4. **Налаштуйте додаткові параметри**
  - Розгорніть відповідні розділи
  - Налаштуйте тільки ті, які вам потрібні



### Типи параметрів у вузлах

- **Текстові поля**
    - Можна ввести текст напряму
    - Або використовувати **вирази** (`{{ ... }}` або через )
  - **Випадаючі списки**
    - Виберіть значення зі списку
    - Деякі списки динамічні — завантажують дані із зовнішніх сервісів
  - **Перемикачі (Toggle)**
    - Вмикають/вимикають окремі функції
  - **Поля масивів/об'єктів**
    - Натисніть "Add Item" для додавання кількох значень
    - Можна створювати вкладені структури
  - **Облікові дані (Credentials)**
    - Виберіть існуючі або створіть нові
    - Дотримуйтесь підказок для підключення до сервісу (OAuth, токени тощо)
- 



## Використання виразів (Expressions)

Вирази дають змогу створювати **динамічні значення** на основі даних із попередніх вузлів.



### Доступ до даних вузлів

- `$item("NodeName").json.field` — дані з конкретного вузла
- `$json.field` — поле з попереднього вузла

**Приклад:**

```
$item("Get Customer").json.name.toUpperCase()
```

**Форматування дати:**

```
$formatDate($item("Get Order").json.createdAt, "YYYY-MM-DD")
```

## Умовне значення (тернарний оператор):

```
$json.status === "active" ? "Активний" : "Архівний"
```

## Вираз з логічним оператором:

```
$json.amount > 1000 || $json.priority === "high"
```

### Редактор виразів

- Натисніть  біля поля → оберіть "Add Expression"
- Використовуйте **панель змінних** для навігації по доступним даним
- Тестуйте вирази до збереження

### Рекомендації з конфігурації

- Надавайте вузлам змістовні назви
  - Наприклад: "Отримати клієнтів" замість "HTTP Request"
  - Клікніть на назву вузла у конфігурації → змініть її
- Тестуйте покроково
  - Використовуйте кнопку "Execute Node" для перевірки кожного вузла
  - Аналізуйте вхідні/вихідні дані до переходу до наступного кроку
- Документуйте складні вузли
  - Використовуйте вкладку "Notes"
  - Описуйте залежності, параметри, зовнішні API



- **Враховуйте помилки**
    - Налаштовуйте поведінку при помилках
    - Вирішіть, чи має воркфлоу продовжуватися, чи зупинятись
- 



## Тестування та налагодження

n8n надає інструменти для перевірки, діагностики й відлагодження воркфлоу.



### Методи тестування

- **Запуск усього воркфлоу**
    - Кнопка "Execute Workflow" у верхній панелі
    - Після запуску — аналіз результатів вузлів і даних
  - **Тестування окремих вузлів**
    - Виберіть вузол
    - Натисніть "Execute Node"
    - Особливо корисно при конфігурації API, обробки даних
  - **Тестування з прикладними даними**
    - У тригер-вузлах (Webhook, Schedule) доступна опція "Test with sample data"
    - Можна змодельовати реальну подію
- 



## Техніки налагодження (Debugging Techniques)

Правильне налагодження допомагає швидко виявляти й усувати помилки у ваших воркфлоу.



### Перевірка даних між вузлами

- Після виконання воркфлоу клікніть на **лінію з'єднання між вузлами**
  - Перегляньте, **які саме дані проходять** між ними
  - Допомагає виявити, де відбувається несподівана трансформація
-

## Використання вузла Debug

- Додайте вузол “Debug” у будь-яку частину воркфлоу
- Налаштуйте його для виведення специфічної інформації
- Зручно для відстеження змінних і порядку виконання

## Функціональні вузли для логування

- Використовуйте Function Node з `console.log()` для відображення даних

```
console.log("Current data:", $input.all()); return $input.all(); // Передає дані далі без змін
```

## Перегляд журналу виконань

- У вкладці “Executions” у головному меню
- Переглядайте статус, повідомлення про помилки, вхідні та вихідні дані
- Корисно для аналізу помилок у продакшн-сценаріях

## Типові помилки та рішення

| Проблема               | Причина                                           | Рішення                                              |
|------------------------|---------------------------------------------------|------------------------------------------------------|
| Відсутні або null-дані | Попередній вузол не повертає очікувані значення   | Перевірте правильність імен полів, шляхів у виразах  |
| Помилки авторизації    | Недійсні облікові дані або відсутні права доступу | Оновіть токени, перевірте права в сервісі            |
| Невірне форматування   | Невідповідність типів даних (рядок, масив, число) | Використовуйте перетворення, видаляйте зайві символи |
| Затримки синхронізації | Зовнішній сервіс ще не оновив дані                | Додайте затримку (Wait Node) або повторну спробу     |

## Поступова розробка (Iterative Approach)

- Починайте з простого
    - Створіть базовий воркфлоу з мінімумом вузлів
  - Ускладнюйте поетапно
    - Додавайте вузли по одному та **тестуйте** після кожної зміни
  - Зберігайте версії
    - Використовуйте "**Workflow History**", щоб зберегти контроль над змінами
    - Поверніться до стабільної версії, якщо щось зламається
  - Ведіть нотатки
    - Документуйте тести, винятки, тимчасові рішення у вкладці "**Notes**" вузлів
- 

## Робота з тригерами (Trigger Options)

Тригери — це вузли, що визначають, коли запускається ваш воркфлоу. n8n підтримує різні типи тригерів під різні задачі.

---

## Ручні тригери (Manual Triggers)

Види ручного запуску:

- Кнопка "Execute Workflow"
    - Функція у верхній панелі
    - Запускає весь воркфлоу вручну
    - Зручно для тестування та одноразових задач
  - Manual Trigger Node
    - Простий вузол-тригер
    - Запускається вручну
    - Може містити **тестові дані**
  - Webhook у ручному режимі
    - Працює у тестовому режимі — дозволяє імітувати виклики API
-

## ⚙️ Налаштування ручного тригера

- Додайте вузол **Manual Trigger** першим у воркфлоу
- У вкладці **"Test Data"** введіть тестові значення, які будуть передані далі

### Приклад:

```
{ "name": "Test User", "email": "test@example.com", "subscription": "premium" }
```

## ▶️ Запуск вручну

1. Оберіть воркфлоу у списку
2. Натисніть **"Execute Workflow"**  
або клікніть правою кнопкою по Manual Trigger → **"Execute Node"**
3. Перегляньте результат виконання вузлів і дані

## 🔄 Приклади використання ручних тригерів

Ручні тригери ідеально підходять для запуску воркфлоу **на вимогу**, коли не потрібен постійний або автоматичний запуск.

### 📌 Типові сценарії:

- **Звіти на вимогу**
  - Генеруйте звіти тільки тоді, коли це потрібно
  - Уникайте зайвого навантаження за рахунок запуску вручну
- **Адміністративні задачі**
  - Ручне обслуговування, очищення бази, запуск імпорту/експорту
  - Пакетна обробка (batch jobs) по запиту

- Тестування та розробка
    - Тестуйте логіку воркфлоу з різними наборами даних
    - Використовуйте ручний тригер під час налаштування та налагодження
- 

## Тригери за розкладом (Scheduled Triggers)

Ці тригери запускають воркфлоу **в автоматичний час**, без участі людини. Використовуються для регулярних дій — наприклад, щоденні звіти чи синхронізація.

---

### Типи тригерів за часом:

- **Schedule Trigger**
    - Основний вузол для запуску по розкладу
    - Підтримує прості шаблони (щодня, щогодини, щотижня тощо)
  - **Cron Trigger**
    - Використовує **cron-вирази** для точного контролю
    - Підходить для досвідчених користувачів або складних графіків
- 

### Налаштування тригерів:

#### **Schedule Trigger:**

- Додайте вузол **Schedule Trigger** як перший
- Оберіть режим:
  - **"Every X"** — вказати інтервал (хвилини, години, дні тощо)
  - **"Custom"** — вибрати конкретні дні та години

#### **Cron Trigger:**

- Додайте **Cron Trigger Node**
- Введіть cron-вираз
  - Наприклад: `0 9 * * 1-5` — кожен будній день о 9:00

- Можна використовувати генератори cron-виразів для зручності
- 

## ✓ Рекомендації:

- Уникайте перевантаження
    - Не запускайте багато воркфлоу одночасно
    - Розкидайте їх у часі, щоб знизити навантаження
  - Обробка пропущених запусків
    - Подумайте, що має відбутись, якщо п8п був недоступний під час розкладу
    - Додайте перевірку дати останнього запуску
  - Перевірка часової логіки
    - Тестуйте, як працює логіка при переході між датами, часовими зонами
    - Використовуйте функції обробки дат: `$formatDate()` , `$now` , `$addMinutes()` , тощо
  - Документуйте графік
    - Напишіть у "Notes", чому вибрано саме такий розклад
    - Вкажіть, чи є залежності від інших запланованих задач
- 

## Тригери Webhook

Webhook — це один з найпотужніших способів запуску воркфлоу зовнішніми системами через HTTP-запити. Підходить для подій з сайтів, CRM, ботів тощо.

---

## Види Webhook тригерів:

- Webhook Node
  - Основний вузол для прийому HTTP-запитів
  - Підтримує GET, POST, PUT, DELETE
  - Може повертати відповідь клієнту

- **Wait Node**

- Призупиняє воркфлоу до отримання webhook-запиту
  - Корисно для **багатокрокових процесів**, де частину обробки виконує інший сервіс
- 

## **Налаштування Webhook:**

1. Додайте вузол **Webhook** як перший
2. Оберіть HTTP-метод (наприклад, **POST**)
3. Вкажіть спосіб автентифікації (або "None")
4. **Активуйте** воркфлоу — згенерується webhook URL

## **Формати URL:**

- n8n Cloud: `https://your-workspace.n8n.cloud/webhook/your-path`
  - Self-hosted: `http://your-server:5678/webhook/your-path`
- 

## **Варіанти автентифікації:**

- **None** — відкритий доступ (небезпечно)
  - **Basic Auth** — логін/пароль
  - **Header Auth** — спеціальний заголовок (наприклад, API-ключ)
  - **Query Auth** — параметри у URL
- 

## **Конфігурація відповіді:**

- Налаштуйте **статус-код** (200, 400, 500 тощо)
  - Вкажіть **заголовки** (Headers) при потребі
  - Визначте **тіло відповіді** (JSON, текст, тощо)
- 

## **Безпека Webhook**

- **Захистіть доступ**
    - Завжди вмикайте автентифікацію
    - Використовуйте заголовки або секрети в URL
  - **Секретність URL**
    - URL має випадкову частину, не публікуйте його відкрито
  - **Перевірка запитів**
    - Обов'язково перевіряйте вхідні дані
    - Переконайтесь, що є всі потрібні поля, правильні формати
    - При потребі — реалізуйте перевірку підписів (signature validation)
  - **Обмеження частоти**
    - Встановіть ліміти запитів
    - Слідкуйте за підозрілою активністю або спамом
- 

## Поширені сценарії використання Webhook

Webhook-и — це надзвичайно гнучкий спосіб запуску воркфлоу через зовнішні події або сервіси.

### Типові сценарії:

- **Надсилання форм**
  - Прийом даних із вебформ (наприклад, Contact Form, Typeform)
  - Автоматичне надсилання листа або збереження даних у базі
- **Платіжні повідомлення**
  - Прийом нотифікацій від платіжних систем (Stripe, PayPal)
  - Оновлення статусу замовлення, запуск процесу доставки
- **Інтеграції з сервісами**
  - Використання webhook-підтримки від GitHub, Notion, Calendly тощо
  - Реакція на створення подій, комітів, оновлень



- Кастомні API
    - Створення власних API-ендпоінтів через Webhook вузол
    - Дає змогу іншим системам взаємодіяти з n8n
  - Інтеграція з IoT
    - Отримання даних від пристроїв (сенсори, контролери)
    - Обробка показників, запис у БД або алерти
- 

## Тестування Webhook тригерів

### Через інтерфейс n8n

- Вебхук вузол має кнопку "Test"
- Дозволяє перевірити структуру запиту, не виходячи з редактора

### Зовнішні інструменти:

- Postman / cURL

```
curl -X POST https://your-n8n-instance.com/webhook/path \ -H "Content-Type: application/json" \ -d '{"name": "Test User", "action": "signup"}'
```

- Webhook-тест сервіси
    - Наприклад: [webhook.site](https://webhook.site)
    - Допомогає візуально перевірити, що саме надсилається у n8n
- 

## Подієві тригери (Event-Based Triggers)

Ці тригери автоматично запускають воркфлоу **при настанні події** в зовнішньому сервісі (новий лист, новий клієнт, зміна статусу задачі тощо).

---

### Типи подієвих тригерів:

- **Спеціалізовані вузли**
    - Працюють з популярними сервісами (Slack, GitHub, Airtable, Notion)
    - Автоматично аутентифікуються та опитують зміни
  - **Універсальні (через Webhook)**
    - Використовуються для інтеграції з будь-яким сервісом, що надсилає HTTP-запити
    - Ви самі конфігуруєте обробку подій
- 

## Найпопулярніші вузли-тригери:

- **Електронна пошта**
    - **IMAP Email:** Тригер на нові листи у вхідних
    - **Gmail Node:** Моніторинг нових листів за критеріями
  - **CRM / Маркетинг**
    - **HubSpot:** Нові контакти, угоди або події
    - **Salesforce:** Створення/оновлення записів
    - **Mailchimp:** Нові підписники, запуск кампаній
  - **Управління проектами**
    - **Asana, Jira, Trello:** Нові задачі, зміни статусу, коментарі
  - **Інструменти розробки**
    - **GitHub / GitLab / Bitbucket:** Нові коміти, pull/merge-запити, CI/CD події
  - **Комунікаційні сервіси**
    - **Slack / Discord / Teams:** Повідомлення, реакції, зміни в каналах
- 

## Налаштування подієвих тригерів

### Аутентифікація

- Додайте тригер
- Під'єднайте обліковий запис через **Credentials**
- Підтвердіть доступ (OAuth або API-ключ)



## Інтервали опитування

- Більшість тригерів працює через **polling** (перевіряють події кожні X хвилин)
- Ви можете налаштувати частоту:
  - Часто (1–5 хв) — швидка реакція
  - Рідше — менше навантаження на API



## Фільтрація подій

- Визначте, на які саме події реагувати
  - Наприклад: тільки нові задачі від певного користувача, тільки pull requests на `main`, тощо
- 



## Приклади з реального життя



### Автоматизація підтримки клієнтів

- Тригер: Новий лист у вхідних
  - Дії:
    - Створити тікет у HelpDesk
    - Відповісти шаблонним листом
    - Призначити агента на основі теми
- 



### DevOps-автоматизація

- Тригер: Новий pull request у GitHub
  - Дії:
    - Повідомити в Slack
    - Створити задачу на перевірку коду в Jira
    - Запустити автотести або CI/CD пайплайн
- 



### Автоматизація в продажах

- Тригер: Створено нову угоду в HubSpot

- Дії:
    - Надіслати повідомлення менеджеру
    - Створити завдання на фоллоу-ап
    - Згенерувати шаблон комерційної пропозиції
- 

## Моніторинг соціальних мереж

Сценарій:

- Тригер: Згадка про компанію в Twitter
  - Дія: Аналіз настрою (sentiment analysis)
  - Дія: Додати в базу клієнтів (якщо доречно)
  - Дія: Повідомити команду (якщо негативна згадка)
- 

## Найкращі практики використання подієвих тригерів

### Обробка дублікатів

- Впроваджуйте **логіку унікальності** подій
- Зберігайте ідентифікатори вже оброблених подій
- Запобігайте повторному запуску однієї й тієї ж події

### Обробка помилок

- Продумайте реакцію на **відмови сервісу** чи зміни в API
- Визначте, що робити у випадку пропущених подій

### Моніторинг використання API

- Знайте обмеження по кількості запитів (rate limits)
- Налаштуйте **інтервали опитування** відповідно до допустимого ліміту

### Тестування

- Створіть **тестові події** у зовнішньому сервісі

- Перевірте, що ваш воркфлоу спрацьовує правильно
  - Тестуйте нестандартні або граничні випадки
- 

## Створення першого робочого воркфлоу: "Щоденний прогноз погоди"

Мета:

Автоматично щодня отримувати прогноз погоди й надсилати листа.

### Що буде робити воркфлоу:

1. Автоматично запускатися щоранку
  2. Отримувати погодні дані з API
  3. Обробляти й формувати ці дані
  4. Надсилати email із прогнозом
- 

### ◆ Крок 1: Створіть новий воркфлоу

- Відкрийте n8n
  - Зліва оберіть **Workflows**
  - Натисніть + **Create** → **Workflow**
  - Назвіть його **"Daily Weather Report"**
  - Додайте опис: *"Надсилає щоденний прогноз погоди електронною поштою"*
- 

### ◆ Крок 2: Додайте тригер "Schedule"

- Натисніть **"Add first step..."**
- Знайдіть та додайте вузол **Schedule**
- Налаштуйте:
  - **Mode:** *Every Day*
  - **Time:** *07:00*
  - **Timezone:** ваша локальна
- Назвіть вузол: **"Daily at 7 AM"**

---

### ◆ Крок 3: Отримайте дані про погоду

- Додайте HTTP Request Node після Schedule
- Назвіть його "Get Weather Data"
- Налаштуйте:
  - Method: GET
  - URL:

```
https://api.openweathermap.org/data/2.5/forecast?q=Kyiv&appid=your_api_key&units=metric
```

🔑 Замість your\_api\_key вставте власний API-ключ з openweathermap.org

- У вкладці Headers додайте:
  - Content-Type : application/json
- Натисніть Execute Node для перевірки відповіді

---

### ◆ Крок 4: Обробіть дані

- Додайте Function Node
- Назвіть: "Process Weather Data"
- Вставте код:

```
const weatherData = $input.all()[0].json; const todayForecast = weatherData.list[0]; const processedData = { city: weatherData.city.name, country: weatherData.city.country, date: new Date(todayForecast.dt * 1000).toLocaleDateString(), temperature: Math.round(todayForecast.main.temp), feels_like: Math.round(todayForecast.main.feels_like), description: todayForecast.weather[0].description, humidity: todayForecast.main.humidity, wind_speed: todayForecast.wind.speed, forecast_summary: `Очікується ${todayForecast.weather[0].description}, температура близько ${Math.round(todayForecast.main.temp)}°C` }; return [{ json: processedData }];
```



## Крок 5: Форматування контенту email

- Додайте вузол Set
  - Клікніть "+" після вузла *Process Weather Data*
  - Оберіть Set

- Налаштування вузла

- Назвіть його: **Format Email Content**
- Натисніть "Add Value" та додайте поля:

- Field: `subject`

Value:

```
Daily Weather Report for {{$json.city}} - {{$json.date}}
```

- Field: `emailContent`

Value (HTML):

```
<h2>Weather Report for {{$json.city}}, {{$json.country}}</h2> <p><strong>Date:</strong> {{$json.date}}</p> <p><strong>Forecast:</strong> {{$json.forecast_summary}}</p> <h3>Details:</h3> <ul> <li><strong>Temperature:</strong> {{$json.temperature}}°C</li> <li><strong>Feels Like:</strong> {{$json.feels_like}}°C</li> <li><strong>Description:</strong> {{$json.description}}</li> <li><strong>Humidity:</strong> {{$json.humidity}}%</li> <li><strong>Wind Speed:</strong> {{$json.wind_speed}} m/s</li> </ul> <p>Have a great day!</p>
```

- Протестуйте вузол — натисніть **Execute Node**, щоб перевірити структуру даних

---

## Крок 6: Надішліть email

- Додайте вузол Send Email
  - Після Set Node натисніть "+" → **Send Email**



- Налаштування вузла
    - Назвіть: **Send Weather Report**
    - Виберіть або створіть email-облікові дані (SMTP або інтеграція з Gmail/SendGrid)
    - **From Email:** ваша електронна адреса
    - **To Email:** адреса одержувача
    - **Subject:** `{{ $json.subject }}`
    - **HTML Content:** `{{ $json.emailContent }}`
  - Протестуйте — натисніть **Execute Node** і перевірте, чи надійшов email
- 

## ✓ Крок 7: Активуйте воркфлоу

- Збережіть воркфлоу – натисніть **Save**
  - Увімкніть – перемкніть "Active" у верхньому правому куті
  - Перевірте активацію – планувальник покаже час наступного запуску
- 

## ⚠ Крок 8: Обробка помилок (необов'язково)

- Створіть окремий воркфлоу
    - Назвіть: **Weather Report Error Handler**
    - Додайте вузол **Webhook** як тригер
    - Додайте дії: надіслати повідомлення про помилку (наприклад, email або Slack)
  - Прив'яжіть до основного воркфлоу
    - Відкрийте основний воркфлоу
    - Клікніть ⚙ у верхній панелі → **Settings**
    - У полі **Error Workflow** виберіть *Weather Report Error Handler*
- 

## Розширення робочого процесу

Після того як базовий робочий процес налаштовано, розгляньте наступні покращення:

- **Додавання кількох локацій**
  - Використовуйте вузол *Split* для обробки кількох міст
  - Об'єднайте результати в одному електронному листі
- **Покращення форматування**
  - Додайте іконки погоди відповідно до умов
  - Включіть прогноз на кілька днів
- **Додавання умов**
  - Використовуйте вузол *IF*, щоб надсилати спеціальні сповіщення про екстремальні погодні умови
  - Персоналізуйте повідомлення залежно від погодних умов
- **Збереження історичних даних**
  - Додайте вузли для збереження погодних даних у базі даних
  - Створюйте тренди та порівняння з часом

Цей покроковий підручник демонструє процес створення базового робочого процесу в n8n. Дотримуючись цих кроків, ви створили практичну автоматизацію, яка щоденно запускається, отримує зовнішні дані, обробляє їх і надсилає корисну інформацію електронною поштою. Цей шаблон можна адаптувати до безлічі інших сценаріїв автоматизації, змінюючи джерело даних, логіку обробки та спосіб виводу.

## Розділ 5: Робота з інтеграціями

### Основи інтеграцій

Інтеграції — це основа функціональності n8n, яка дозволяє з'єднувати та автоматизувати робочі процеси між різними застосунками та сервісами. Щоб створювати потужні рішення з автоматизації, важливо вміти ефективно працювати з інтеграціями.

### Огляд вбудованих вузлів

n8n має велику бібліотеку вбудованих вузлів, які забезпечують готові до використання інтеграції з популярними сервісами та функціоналом.

## Категорії вузлів

Вбудовані вузли в n8n поділені на кілька категорій, щоб вам було легше знаходити потрібні:

- **Основні вузли (Core Nodes)**
  - Основна функціональність для керування робочим процесом і обробки даних
  - Приклади: HTTP Request, Function, IF, Switch, Merge, Set
- **Вузли комунікації (Communication Nodes)**
  - Сервіси електронної пошти, платформи для обміну повідомленнями та інструменти сповіщень
  - Приклади: Gmail, Slack, Discord, Telegram, Twilio
- **CRM та продажі (CRM & Sales Nodes)**
  - Автоматизація продажів і управління взаєминами з клієнтами
  - Приклади: HubSpot, Salesforce, Pipedrive, Zoho CRM
- **Бази даних (Database Nodes)**
  - Підключення до баз даних і робота з ними
  - Приклади: MySQL, PostgreSQL, MongoDB, Airtable, Supabase
- **Розробка (Development Nodes)**
  - Інструменти для розробників і систем керування кодом
  - Приклади: GitHub, GitLab, Bitbucket, Jira
- **Маркетинг (Marketing Nodes)**
  - Автоматизація маркетингу та аналітика
  - Приклади: Mailchimp, ActiveCampaign, Google Analytics
- **Продуктивність (Productivity Nodes)**
  - Управління завданнями й обробка документів
  - Приклади: Google Drive, Notion, Asana, Trello

- **Фінанси (Finance Nodes)**
  - Платіжні системи та фінансові сервіси
  - Приклади: Stripe, PayPal, QuickBooks
- **Штучний інтелект і ML (AI & Machine Learning Nodes)**
  - Сервіси зі штучним інтелектом і машинним навчанням
  - Приклади: OpenAI, LangChain, HuggingFace

## Типи вузлів

У межах кожної категорії вузли також поділяються за функціоналом:

- **Вузли-тригери (Trigger Nodes)**
  - Запускають робочі процеси на основі подій або розкладу
  - Приклади: Webhook, Schedule, Gmail Trigger
- **Вузли-дії (Action Nodes)**
  - Виконують операції з зовнішніми сервісами
  - Приклади: Slack (відправка повідомлення), Google Sheets (додавання рядка)
- **Допоміжні вузли (Helper Nodes)**
  - Обробка й трансформація даних
  - Приклади: Set, Function, Split In Batches
- **Керування потоком (Flow Control Nodes)**
  - Керує логікою виконання робочого процесу
  - Приклади: IF, Switch, Merge, Wait

## Структура вузла

Кожен вузол у n8n має однакову базову структуру:

- **Вхідні дані (Input):** отримує дані з попереднього вузла
- **Конфігурація (Configuration):** налаштування, характерні для конкретного вузла
- **Обробка (Processing):** внутрішні дії, що виконуються над даними
- **Вихідні дані (Output):** дані, які передаються наступному вузлу

Розуміння цієї структури допомагає передбачати поведінку вузлів і логіку передачі даних у робочому процесі.

## Пошук потрібного вузла

У n8n доступні сотні вузлів, тому вміння швидко знаходити потрібні — ключ до ефективного створення процесів.

## Стратегії пошуку

- **Пошук за ключовими словами**
    - Використовуйте панель пошуку у верхній частині меню вузлів
    - Вводьте назви сервісів, дій або функцій
    - Приклади: "email", "database", "notification"
  - **Перегляд категорій**
    - Розгорніть категорії в панелі вузлів
    - Переглядайте пов'язані вузли
    - Корисно для вивчення доступних інтеграцій
  - **Нещодавно використані**
    - Перегляньте розділ "Recently Used", щоб швидко знайти вузли, які вже використовувалися
  - **Рекомендації спільноти**
    - Відвідайте форум спільноти n8n для рекомендацій щодо вузлів для певних завдань
    - Вивчайте приклади робочих процесів, якими діляться інші користувачі
- 

## Оцінка варіантів вузлів

Коли кілька вузлів потенційно підходять для ваших завдань, враховуйте наступні фактори:

- **Вузли "рідні" чи загальні**
    - Рідні вузли (наприклад, "Slack") мають спеціалізовану функціональність для конкретних сервісів
    - Загальні вузли (наприклад, "HTTP Request") можуть працювати з будь-яким сервісом, який має API
    - Рідні вузли зазвичай легші в налаштуванні, але можуть мати обмежену кількість операцій
  - **Покриття операцій**
    - Перевірте, чи підтримує вузол саме ті дії, які вам потрібні
    - Деякі вузли охоплюють лише базові операції, інші мають повний функціонал
  - **Методи автентифікації**
    - Переконайтесь, що вузол підтримує бажаний метод автентифікації
    - Деякі вузли підтримують кілька способів (OAuth, API Key тощо)
  - **Частота оновлень**
    - Перевірте, коли вузол востаннє оновлювався
    - Часто оновлювані вузли зазвичай надійніші та мають більше функцій
  - **Спільнота vs. офіційна підтримка**
    - Вбудовані вузли офіційно підтримуються командою n8n
    - Вузли зі спільноти надають додаткові можливості, але можуть відрізнятися за якістю та підтримкою
- 

## Документація до вузлів

Кожен вузол у n8n має супровідну документацію, яка допомагає зрозуміти його призначення та можливості:

- **Вбудована документація**
  - Натисніть іконку "i" у вузлі, щоб переглянути документацію
  - Містить описи операцій та параметрів
  - Часто включає приклади поширених сценаріїв використання

- **Офіційна документація n8n**
    - Повна документація доступна на сайті [docs.n8n.io](https://docs.n8n.io)
    - Містить детальні гіді та приклади для кожного вузла
    - Регулярно оновлюється з новими функціями
  - **Документація з автентифікації**
    - Інструкції щодо налаштування доступу до зовнішніх сервісів
    - Покрокове отримання API-ключів або токенів
    - Вирішення поширених проблем з авторизацією
- 

## Автентифікація та облікові дані

Більшість інтеграцій вимагають автентифікації для безпечного доступу до зовнішніх сервісів. n8n має потужну систему управління обліковими даними для підтримки різних методів автентифікації.

---

## Типи облікових даних

n8n підтримує кілька способів автентифікації:

- **API Key**
  - Простий метод на основі ключа
  - Зазвичай вимагає лише одного токена або ключа
  - Приклад: погодні API, прості REST-сервіси
- **OAuth2**
  - Захищена автентифікація на основі токена без передачі паролів
  - Використовує редирект до сервісу для авторизації
  - Приклад: сервіси Google, Microsoft 365, соціальні мережі
- **Ім'я користувача/пароль**
  - Базова автентифікація з логіном і паролем
  - Застосовується до сервісів без токен-підтримки
  - Приклад: SMTP пошта, деякі бази даних

- **JWT (JSON Web Token)**
    - Автентифікація на основі підписаних токенів
    - Використовується в багатьох сучасних API
    - Приклад: кастомні API, платформи SaaS
  - **Специфічна автентифікація**
    - Індивідуальні методи для конкретних сервісів
    - Може вимагати кілька полів або спеціальні налаштування
    - Приклад: AWS (ключ доступу + секрет + регіон)
- 

## Створення та керування обліковими даними

Щоб налаштувати облікові дані в n8n:

- **Через налаштування вузла**
    - Під час конфігурації вузла, який потребує автентифікації
    - Оберіть "Create New" у випадяючому списку облікових даних
    - Введіть необхідну інформацію
  - **Через розділ "Credentials"**
    - Відкрийте меню "Credentials" на бічній панелі
    - Натисніть "Create", щоб додати нові облікові дані
    - Оберіть тип і заповніть поля
  - **Зберігання облікових даних**
    - Облікові дані шифруються перед збереженням
    - Конфіденційна інформація ніколи не зберігається у вигляді відкритого тексту в JSON робочого процесу
    - У self-hosted версіях доступні додаткові опції шифрування
- 

## Процес автентифікації OAuth2

Для автентифікації через OAuth2:



- **Запуск процесу OAuth**
    - Натисніть "Create New" при виборі облікових даних
    - Оберіть опцію OAuth2 для потрібного сервісу
  - **Авторизація**
    - n8n перенаправить вас на сторінку входу сервісу
    - Наддайте дозвіл n8n на доступ до вашого облікового запису
  - **Обмін токенів**
    - Сервіс надає авторизаційний код
    - n8n обмінює його на access та refresh токени
    - Токени зберігаються безпечно для подальшого використання
  - **Оновлення токенів**
    - n8n автоматично оновлює токени після закінчення їх дії
    - Повторна авторизація потрібна лише при зміні дозволів
- 

## Найкращі практики безпеки облікових даних

Щоб забезпечити безпечну інтеграцію:

- **Використовуйте окремі сервісні акаунти**
  - Створюйте спеціальні облікові записи лише для автоматизації
  - Обмежуйте права доступу тільки необхідними
- **Регулярно перевіряйте облікові дані**
  - Видаляйте непотрібні або застарілі облікові дані
  - Оновлюйте дані після зміни команди чи політик
- **Використовуйте змінні середовища (env variables)**
  - Зберігайте конфіденційні значення у змінних середовища
  - Посилайтесь на них у налаштуваннях облікових даних
- **Обмеження за IP-адресами**
  - По можливості обмежуйте доступ до API для конкретних IP-адрес
  - Це додатковий рівень безпеки поверх токенів

- **Моніторинг використання**
    - Відстежуйте, які робочі процеси використовують які облікові дані
    - Слідкуйте за підозрілою або нестандартною активністю
- 

## Тестування з'єднань

Перш ніж створювати складні робочі процеси, важливо переконатися, що ваші інтеграції працюють належним чином. n8n пропонує кілька способів тестування з'єднань із зовнішніми сервісами.

### Базове тестування з'єднання

- **Перевірка облікових даних**
    - Більшість типів облікових даних мають кнопку "Test"
    - Перевіряє правильність введених параметрів і доступність сервісу
  - **Виконання простого вузла**
    - Додайте вузол із базовою операцією (наприклад, "List Records")
    - Налаштуйте мінімальні параметри
    - Запустіть вузол, щоб перевірити з'єднання й отримання даних
  - **Тестування через HTTP Request**
    - Використовуйте вузол HTTP Request для ручного тестування API
    - Перевірте коди відповіді та формат отриманих даних
    - Особливо корисно для усунення помилок у налаштуванні
- 

## Вирішення проблем зі з'єднанням

Поширені проблеми та їх вирішення:

- **Помилки автентифікації**
  - Перевірте правильність введених облікових даних
  - Переконайтеся, що токени не протерміновані
  - Перевірте, чи має акаунт необхідні дозволи
  - Уникайте помилок у ключах чи секретах API

- **Мережеві проблеми**
    - Перевірте, чи доступний сервіс з вашого середовища n8n
    - Переконайтесь, що міжмережевий екран (firewall) не блокує підключення
    - Для self-hosted версій перевірте мережеву інфраструктуру
  - **Обмеження по частоті запитів (rate limits)**
    - Багато API мають ліміти на кількість запитів
    - За потреби додайте затримку між запитами
    - Уважно читайте повідомлення про помилки з API
  - **Зміни в API**
    - Сервіси можуть змінювати структуру або правила API
    - Перевірте офіційні повідомлення про оновлення
    - Оновіть конфігурацію вузлів згідно з новими вимогами
- 

## Найкращі практики тестування з'єднань

- **Починайте з простого**
    - Почніть з найпростішої операції
    - Ускладнюйте логіку лише після успішного тесту
  - **Перевіряйте кожен тип операції**
    - Тестуйте всі типи дій, які плануєте використовувати
    - Для деяких з них можуть вимагатися додаткові дозволи
  - **Документуйте конфігурації**
    - Ведіть нотатки про налаштування, API, обхідні рішення
    - Це допоможе під час налагодження або передачі проєкту
  - **Регулярний моніторинг**
    - Періодично перевіряйте стабільність критичних інтеграцій
    - Налаштуйте сповіщення при помилках з'єднання
- 

Розуміння цих принципів інтеграцій допоможе вам впевнено знаходити, налаштовувати та тестувати підключення, які є основою для автоматизації у n8n.

---

# Популярні категорії інтеграцій

n8n підтримує широкий спектр інтеграцій у різних категоріях. У цьому розділі розглянуто найпоширеніші типи інтеграцій і надано поради щодо ефективного використання їх у ваших робочих процесах.

---

## CRM та маркетингові інструменти

Системи управління взаєминами з клієнтами (CRM) та інструменти автоматизації маркетингу — одні з найчастіше використовуваних інтеграцій у n8n.

### Популярні CRM-інтеграції

- **HubSpot**
  - **Типові дії:**
    - Створення/оновлення контактів, компаній і угод
    - Запуск робочих процесів при додаванні нових записів
    - Відстеження змін етапів угод
  - **Приклад сценарію:** Автоматичне створення завдань, коли угода переходить на певний етап
- **Salesforce**
  - **Типові дії:**
    - Управління лідами, контактами, акаунтами й можливостями
    - Створення об'єктів і записів
    - Запити даних через SOQL
  - **Приклад сценарію:** Синхронізація клієнтських даних між Salesforce та іншими системами

- **Pipedrive**
    - **Типові дії:**
      - Керування угодами та активностями
      - Відстеження просування по воронці продажів
      - Оновлення контактної інформації
    - **Приклад сценарію:** Створення завдань-нагадувань після зміни статусу угоди
- 

## Інтеграції для автоматизації маркетингу

- **Mailchimp**
    - **Типові дії:**
      - Керування підписниками та списками
      - Створення та надсилання кампаній
      - Відстеження залучення (open rate, click rate)
    - **Приклад сценарію:** Додавання нових клієнтів до відповідних розсилок на основі історії покупок
  - **SendGrid**
    - **Типові дії:**
      - Надсилання транзакційних листів
      - Управління списками контактів
      - Відстеження доставки листів і взаємодії з ними
    - **Приклад сценарію:** Автоматичне надсилання персоналізованих підтверджень замовлення та оновлень доставки
- 

## Платформи комунікації

Інструменти для комунікації важливі для співпраці в команді та взаємодії з клієнтами. Інтеграція таких платформ у п8п дає змогу створювати автоматизовані сповіщення та інтерактивні робочі процеси.

## Популярні інтеграції для комунікації

- **Slack**
    - **Типові дії:**
      - Надсилання повідомлень у канали або конкретним користувачам
      - Створення/оновлення тредів
      - Реакція на повідомлення та події
    - **Приклад сценарію:** Автоматичне сповіщення про критичні помилки системи
  - **Microsoft Teams**
    - **Типові дії:**
      - Надсилання повідомлень у канали
      - Створення завдань і управління ними
      - Запланування зустрічей
    - **Приклад сценарію:** Щоденні оновлення статусу проєкту в каналі команди
  - **Discord**
    - **Типові дії:**
      - Надсилання повідомлень у канали
      - Управління ролями та дозволами
      - Створення інтерактивних повідомлень з embed-форматом
    - **Приклад сценарію:** Модерація спільноти та автоматичне реагування на дії користувачів
  - **Telegram**
    - **Типові дії:**
      - Надсилання повідомлень і сповіщень
      - Створення та керування ботами
      - Обробка команд користувачів
    - **Приклад сценарію:** Бот підтримки, що відповідає на типові запитання клієнтів
- 

## Інтеграції з електронною поштою

- **Gmail**
    - Типові дії:
      - Надсилання та отримання листів
      - Керування мітками та папками
      - Пошук конкретних повідомлень
    - Приклад сценарію: Автоматична обробка вхідних запитів від клієнтів
  - **SMTP/IMAP**
    - Типові дії:
      - Надсилання листів через SMTP
      - Моніторинг вхідної пошти через IMAP
      - Обробка вкладень із листів
    - Приклад сценарію: Обробка пошти з будь-якого провайдера
  - **Outlook/Office 365**
    - Типові дії:
      - Керування листами та календарем
      - Створення й призначення завдань
      - Запланування зустрічей
    - Приклад сценарію: Автоматизація планування зустрічей та надсилання нагадувань
- 

## Інтеграції SMS і голосового зв'язку

- **Twilio**
  - Типові дії:
    - Надсилання та отримання SMS
    - Здійснення та приймання дзвінків
    - Верифікація номерів телефону
  - Приклад сценарію: Надсилання SMS-нагадувань про запис на прийом

- **MessageBird**
    - **Типові дії:**
      - Надсилання SMS та повідомлень WhatsApp
      - Верифікація номерів телефону
      - Керування розмовами
    - **Приклад сценарію:** Багатоканальні сповіщення для клієнтів
- 

## Стратегії інтеграції платформ комунікації

- **Системи сповіщень**
    - Надсилайте повідомлення про важливі події
    - Перенаправляйте сповіщення у відповідні канали
    - Форматуйте повідомлення для легкого сприйняття та дії
  - **Інтерактивні робочі процеси**
    - Створюйте ботів, що реагують на команди
    - Реалізуйте погоджувальні процеси через месенджери
    - Збирайте інформацію через чат-інтерфейси
  - **Оркестрація комунікацій**
    - Обирайте канал зв'язку залежно від терміновості й контексту
    - Ескалуйте повідомлення через інші канали за відсутності відповіді
    - Зберігайте історію спілкування між платформами
  - **Автоматизація командної роботи**
    - Автоматизуйте регулярні оновлення
    - Збирайте та розповсюджуйте інформацію
    - Координуйте дії між командами
- 

## Системи баз даних

Бази даних — це основа для зберігання, отримання та обробки структурованих даних. n8n має потужні інтеграції з різними типами баз даних.

---



## Інтеграції з реляційними (SQL) базами даних

- MySQL
    - Типові дії:
      - Виконання SQL-запитів і збережених процедур
      - Вставка, оновлення та видалення записів
      - Обробка результатів запитів
    - Приклад сценарію: Збереження історії взаємодій із клієнтом для звітності
  - PostgreSQL
    - Типові дії:
      - Виконання складних запитів
      - Робота з JSON-полями
      - Керування транзакціями
    - Приклад сценарію: Ведення аудиту дій користувачів у системі
  - Microsoft SQL Server
    - Типові дії:
      - Виконання T-SQL запитів
      - Управління записами
      - Використання збережених процедур
    - Приклад сценарію: Інтеграція з корпоративними системами
  - SQLite
    - Типові дії:
      - Робота з локальною базою даних
      - Виконання SQL-запитів
      - Керування файлами бази даних
    - Приклад сценарію: Легка система зберігання для простих застосунків
- 

## Інтеграції з NoSQL базами даних

- **MongoDB**
    - Типові дії:
      - Запити до документів і їх зміна
      - Робота з колекціями
      - Агрегаційні операції
    - Приклад сценарію: Зберігання та аналіз напівструктурованих даних
  - **Couchbase**
    - Типові дії:
      - Робота з JSON-документами
      - Виконання N1QL-запитів
      - Робота з bucket-ами та scopes
    - Приклад сценарію: Побудова гнучких моделей даних для складних застосунків
  - **Redis**
    - Типові дії:
      - Робота з ключ-значення
      - Робота зі списками, множинами, хешами
      - Реалізація кешування
    - Приклад сценарію: Кешування з високою продуктивністю, аналітика в реальному часі
- 

## Хмарні сервіси баз даних

- **Airtable**
  - Типові дії:
    - Керування записами в базах
    - Робота з пов'язаними записами
    - Використання представлень (views) і фільтрів
  - Приклад сценарію: Спільне управління проєктами та відстеження завдань

- **Supabase**
    - Типові дії:
      - Виконання PostgreSQL-запитів
      - Управління автентифікацією
      - Робота з реальними підписками на зміни (real-time subscriptions)
    - Приклад сценарію: Побудова сучасних вебдодатків із реального часу
- 

## Firestore

- Типові дії:
    - Читання й запис у Firestore
    - Керування автентифікацією
    - Робота з базою даних у реальному часі
  - Приклад сценарію: Бекенд для мобільного додатку й управління користувачами
- 

## Стратегії інтеграції баз даних

- **Сховище даних (Data Warehousing)**
  - Консолідація даних із кількох джерел
  - Трансформація й нормалізація даних
  - Підготовка до аналітики та звітів
- **Операції CRUD**
  - Створення стандартизованих робочих процесів для створення, читання, оновлення й видалення записів
  - Реалізація валідації та обробки помилок
  - Підтримка цілісності даних між системами
- **Міграція даних**
  - Перенесення даних між різними типами баз даних
  - Трансформація форматів під час перенесення
  - Перевірка цілісності після міграції

- **Кешування та продуктивність**
    - Використання кешу для часто запитуваних даних
    - Оптимізація продуктивності запитів
    - Планування важких операцій на періоди з низьким навантаженням
- 

## Служби зберігання файлів

Інтеграції для роботи з файлами дозволяють керувати документами, зображеннями та іншими файлами в різних хмарних платформах.

---

### Хмарні сервіси зберігання

- **Google Drive**
  - **Типові дії:**
    - Завантаження та завантаження файлів
    - Керування папками та правами доступу
    - Конвертація форматів документів
  - **Приклад сценарію:** Автоматичне резервне копіювання важливих документів
- **Dropbox**
  - **Типові дії:**
    - Завантаження/вивантаження файлів
    - Спільний доступ до папок і файлів
    - Відстеження змін у файлах
  - **Приклад сценарію:** Збір і організація файлів із різних джерел

- **OneDrive / SharePoint**
    - Типові дії:
      - Управління файлами й папками
      - Налаштування прав доступу
      - Робота з бібліотеками документів
    - Приклад сценарію: Робочий процес затвердження та публікації документів
  - **Amazon S3**
    - Типові дії:
      - Завантаження/вивантаження об'єктів
      - Керування bucket-ами та правами
      - Налаштування метаданих і класів зберігання
    - Приклад сценарію: Масштабоване зберігання медіа та розповсюдження
- 

## Інтеграції для керування документами

- **Box**
  - Типові дії:
    - Завантаження й отримання файлів
    - Керування папками та співпрацею
    - Робота з метаданими файлів
  - Приклад сценарію: Безпечний обмін документами між командами
- **DocuSign**
  - Типові дії:
    - Створення та надсилання пакетів для підпису
    - Відстеження статусу підписання
    - Завантаження підписаних документів
  - Приклад сценарію: Автоматизація процесу підписання контрактів

- **Adobe PDF Services**
    - Типові дії:
      - Створення й редагування PDF
      - Витяг тексту та даних із PDF
      - Об'єднання та розділення PDF-файлів
    - Приклад сценарію: Генерація та обробка стандартних форм
- 

## Стратегії інтеграції для роботи з файлами

- **Конвеєри обробки документів**
    - Автоматичне створення та форматування документів
    - Витягування даних з файлів
    - Надсилання на перевірку чи затвердження
  - **Управління медіа**
    - Обробка та оптимізація зображень і відео
    - Створення мініатюр та попереднього перегляду
    - Класифікація медіа за допомогою метаданих
  - **Резервне копіювання та архівація**
    - Регулярне копіювання критично важливих файлів
    - Впровадження політик зберігання та видалення
    - Дотримання регуляцій щодо зберігання даних
  - **Синхронізація файлів**
    - Уніфікація файлів між платформами
    - Вирішення конфліктів при оновленні
    - Ведення історії змін і версій
- 

## Інструменти для розробників

Інтеграції з інструментами розробки допомагають автоматизувати процеси створення програмного забезпечення — від управління кодом до розгортання й моніторингу.

---

## Інтеграції систем контролю версій

- **GitHub**
    - Типові дії:
      - Відстеження подій у репозиторіях
      - Створення та керування задачами (issues)
      - Запуск робочих процесів на основі комітів або pull-запитів
    - Приклад сценарію: Автоматизовані перевірки коду й тестування
  - **GitLab**
    - Типові дії:
      - Керування merge-запитами
      - Відстеження статусу CI/CD pipeline
      - Створення та оновлення задач
    - Приклад сценарію: Безперервна інтеграція та розгортання
  - **Bitbucket**
    - Типові дії:
      - Моніторинг подій у репозиторіях
      - Керування pull-запитами
      - Робота з задачами та коментарями
    - Приклад сценарію: Автоматичне оновлення документації після змін у коді
- 

## Інтеграції управління проєктами

- **Jira**
    - **Типові дії:**
      - Створення та оновлення задач
      - Перехід між статусами задач
      - Відстеження прогресу проєктів
    - **Приклад сценарію:** Синхронізація завдань розробки між різними системами
  - **Linear**
    - **Типові дії:**
      - Управління задачами та багами
      - Відстеження циклів проєкту
      - Зміна статусів задач
    - **Приклад сценарію:** Автоматизоване відстеження помилок та їх усунення
  - **Asana**
    - **Типові дії:**
      - Створення та керування завданнями
      - Організація проєктів та розділів
      - Відстеження виконання завдань
    - **Приклад сценарію:** Координація міжфункціональної роботи команд
- 

## Інтеграції CI/CD та DevOps

- **Jenkins**
  - **Типові дії:**
    - Запуск збірок і джобів
    - Моніторинг статусу збірки
    - Обробка артефактів
  - **Приклад сценарію:** Оркестрація складних процесів збірки й розгортання



- **CircleCI**
    - **Типові дії:**
      - Запуск і моніторинг робочих процесів
      - Обробка результатів збірки
      - Виконання етапів розгортання
    - **Приклад сценарію:** Автоматизоване тестування та деплой
  - **Docker**
    - **Типові дії:**
      - Керування контейнерами та образами
      - Моніторинг стану контейнерів
      - Виконання команд у контейнерах
    - **Приклад сценарію:** Автоматичне налаштування середовищ розробки
- 

## Стратегії інтеграції інструментів розробки

- **Автоматизація робочого процесу розробки**
  - Автоматизуйте повторювані завдання
  - Забезпечте дотримання стандартів коду
  - Спростуйте процеси рев'ю
- **Синхронізація між інструментами**
  - Підтримуйте синхронізацію задач і статусів між платформами
  - Оновлюйте документацію відповідно до змін у коді
  - Забезпечте єдине джерело правди про статус проєкту
- **Управління релізами**
  - Координуйте дії команд під час релізу
  - Генеруйте нотатки релізів автоматично
  - Відстежуйте готовність функцій і виправлень

- **Сповіщення для розробників**
    - Сповіщайте про критичні помилки
    - Повертайте фідбек щодо збірок і тестів
    - Автоматизуйте погодження й перевірку
- 

Розуміння цих популярних категорій інтеграцій і сценаріїв використання дозволяє ефективніше застосовувати можливості n8n для з'єднання ваших ключових інструментів і сервісів.

---

## Робота з API

Хоч n8n і надає вузли для багатьох популярних сервісів, вам часто потрібно буде працювати з API безпосередньо. Вузол **HTTP Request** — це потужний інструмент, який дозволяє підключатись практично до будь-якого REST API.

---

### Вузол HTTP Request: поглиблений розгляд

HTTP Request — один із найуніверсальніших вузлів у n8n, що забезпечує комунікацію з будь-яким сервісом, який підтримує HTTP API.

---

### Базова конфігурація

- **Вибір HTTP-методу**
  - **GET**: Отримання даних з API
  - **POST**: Створення нових ресурсів або надсилання даних
  - **PUT**: Повне оновлення існуючих ресурсів
  - **PATCH**: Часткове оновлення існуючих ресурсів
  - **DELETE**: Видалення ресурсів
  - **HEAD**: Отримання лише заголовків
  - **OPTIONS**: Отримання підтримуваних методів і опцій

- **Налаштування URL**
  - Введіть повну URL-адресу API-ендпоінту
  - Використовуйте вирази для динамічного формування URL
  - Приклад: `https://api.example.com/users/{{json.userId}}`
- **Параметри запиту (Query Parameters)**
  - Додавайте пари ключ-значення до URL
  - Вони автоматично кодуються й додаються до запиту
  - Зручно для фільтрації, сортування та розбивки на сторінки
- **Налаштування заголовків (Headers)**
  - Встановлюйте заголовки HTTP-запиту
  - Поширені заголовки:
    - **Content-Type:** Вказує формат тіла запиту
    - **Authorization:** Дані для авторизації
    - **Accept:** Бажаний формат відповіді
- **Тіло запиту (Request Body)**
  - Вказується при POST, PUT, PATCH
  - Підтримуються різні формати:
    - JSON (найпоширеніший)
    - Form-Encoded
    - Form-Data (для завантаження файлів)
    - Raw (текст, XML тощо)

- **Автентифікація**
    - Оберіть метод автентифікації:
      - Basic Auth
      - Bearer Token
      - Digest Auth
      - OAuth2
      - API Key (в заголовку або запиті)
    - Створіть або оберіть облікові дані
- 

## Розширені можливості

- **Обробка відповіді**
    - **JSON/RAW:** Вибір формату обробки
    - **Повна відповідь:** Включає заголовки, статус-код і тіло
    - **Формат відповіді:** Автоматичне визначення або ручне (JSON, текст)
  - **Обробка помилок**
    - Налаштуйте, які відповіді вважати помилковими
    - Можливість вказати статус-коди (4xx, 5xx)
    - Обробка помилок через вузли Function
  - **Додаткові опції запиту**
    - **Follow Redirects:** Автоматично переходити за редиректами
    - **Ignore SSL Issues:** Ігнорування проблем із SSL-сертифікатом
    - **Timeout:** Встановлення часу очікування відповіді
    - **Proxy:** Надсилання запиту через проксі
  - **Обробка пагінації**
    - Автоматична обробка багатосторінкових відповідей
    - Налаштування параметрів пагінації
    - Можна отримати всі сторінки або встановити обмеження
-

## Типові сценарії використання

- **Інтеграція з API**
    - Підключення до сервісів без окремих вузлів
    - Реалізація користувацьких API-запитів
    - Робота з внутрішніми або приватними API
  - **Отримання даних**
    - Завантаження інформації з зовнішніх джерел
    - Моніторинг змін у сервісах
    - Агрегація даних з кількох ендпоінтів
  - **Webhooks і зворотні виклики**
    - Надсилання даних на webhook-и
    - Реалізація callback-механізмів
    - Повідомлення сторонніх систем про події
  - **Користувацькі операції**
    - Розширення функціональності стандартних вузлів
    - Реалізація функцій, яких немає в наявних інтеграціях
    - Створення повністю кастомних API-робочих процесів
- 

## Поняття RESTful API

Розуміння принципів RESTful API допомагає ефективніше працювати з API у n8n.

---

## Основні принципи REST

- **Ресурси та ендпоінти**
  - API побудовані навколо ресурсів (наприклад, користувачі, товари)
  - Ресурси доступні через URL-адреси (ендпоінти)
  - Приклад: `/users` — список усіх користувачів

- **Методи HTTP і CRUD**

- **CREATE:** `POST` — створення нового ресурсу
  - **READ:** `GET` — отримання ресурсу або колекції
  - **UPDATE:** `PUT` або `PATCH` — оновлення ресурсу
  - **DELETE:** `DELETE` — видалення ресурсу
- 

## Коди статусу HTTP

- **2xx: Успіх**

- `200 OK` : Запит виконано успішно
- `201 Created` : Створено новий ресурс
- `204 No Content` : Успішно, але без тіла відповіді

- **3xx: Перенаправлення**

- `301 Moved` : Постійне перенаправлення
- `304 Not Modified` : Ресурс не змінено (кеш)

- **4xx: Помилка клієнта**

- `400 Bad Request` : Некоректний запит
- `401 Unauthorized` : Неавторизований доступ
- `404 Not Found` : Ресурс не знайдено

- **5xx: Помилка сервера**

- `500 Internal Server Error` : Внутрішня помилка сервера
  - `503 Service Unavailable` : Сервіс тимчасово недоступний
- 

## Ідемпотентність (Idempotency)

- `GET`, `PUT` та `DELETE` повинні бути ідемпотентними — однаковий результат незалежно від кількості викликів
  - `POST` зазвичай не є ідемпотентним — створює новий ресурс кожного разу
-

# Документація API

Більшість API надають документацію, яка включає:

- **Опис ендпоінтів**
    - Перелік доступних ендпоінтів і їх призначення
    - Обов'язкові та необов'язкові параметри
    - Формати відповідей
  - **Вимоги до автентифікації**
    - Як авторизувати запити
    - Як отримати облікові дані
    - Термін дії токенів і процедура оновлення
  - **Інформація про ліміти**
    - Кількість дозволених запитів за певний період
    - Наслідки перевищення
    - Заголовки для відстеження використання
  - **Приклади запитів і відповідей**
    - Приклади коду для типових операцій
    - Очікувані структури відповідей
    - Формати помилок
- 

## Робота з документацією API в n8n

- **Переклад документації у вузли HTTP Request**
  - Відобразіть ендпоінти API у конфігурації вузлів
  - Перетворіть приклади з `curl` на параметри вузлів
  - Додайте автентифікацію згідно з вимогами
- **Тестування ендпоінтів**
  - Використовуйте HTTP Request вузол для перевірки
  - Перевіряйте формат і структуру відповіді
  - З'ясовуйте обов'язкові параметри та заголовки

- **Обробка версій API**

- Додавайте інформацію про версію в URL або заголовки
  - Слідкуйте за змінами, які можуть порушити сумісність
  - Документуйте, яку версію API використовуєте
- 

## Методи автентифікації

Різні API використовують різні методи автентифікації. Їх розуміння — ключ до успішної інтеграції.

---

### API Key

- **Як працює:**
    - Простий ключ або токен додається до кожного запиту
    - Може бути в заголовку, параметрах запиту або тілі
  - **Реалізація в n8n:**
    - Створіть облікові дані "API Key"
    - Вкажіть назву заголовка та значення
    - Оберіть спосіб вставлення (header, query тощо)
  - **Приклад налаштування:**
    - Назва заголовка: `X-API-Key`
    - Значення: `your_api_key_here`
- 

### Basic Authentication

- **Як працює:**
  - Ім'я користувача і пароль кодуються в Base64 і передаються в заголовку Authorization
  - Формат: `Authorization: Basic base64(username:password)`



- **Реалізація в n8n:**
    - Створіть облікові дані типу "Basic Auth"
    - Введіть ім'я та пароль
    - n8n автоматично кодує дані
  - **Захист:**
    - Використовуйте тільки через HTTPS
    - Дані передаються в кожному запиті
    - Краще застосовувати безпечніші методи, якщо доступні
- 

## OAuth 2.0

- **Як працює:**
  - Багатокроковий процес із використанням токенів
  - Розділяє автентифікацію й авторизацію
  - Використовує refresh токени для поновлення доступу
- **Поширені потоки OAuth 2.0:**
  - **Authorization Code** – найпоширеніший для веб-додатків
  - **Client Credentials** – для взаємодії сервер-сервер
  - **Implicit** – для браузерних застосунків (вже менш рекомендований)
  - **Password** – обмін логіном на токен (застарілий метод)
- **Реалізація в n8n:**
  - Створіть облікові дані типу "OAuth2 API"
  - Налаштуйте URL авторизації, токenu та scope
  - Пройдіть процес авторизації
  - n8n автоматично управляє токенами

- Приклад конфігурації OAuth2:

- Authorization URL: `https://api.service.com/oauth/authorize`
  - Token URL: `https://api.service.com/oauth/token`
  - Scope: `read write`
  - Client ID: `your_client_id`
  - Client Secret: `your_client_secret`
- 

## Аутентифікація за допомогою Bearer Token

- Як працює:

- Токен додається до заголовка `Authorization`
- Формат: `Authorization: Bearer your_token_here`
- Часто використовується з JWT (JSON Web Token)

- Реалізація в n8n:

- Створіть облікові дані типу "Bearer Token"
- Введіть значення токена
- n8n автоматично додасть заголовок до запиту

- Керування токенами:

- Деякі токени мають термін дії
  - Може знадобитися окремий робочий процес для отримання нового токена
  - Рекомендується використовувати OAuth2 для автоматичного оновлення
- 

## Користувацька автентифікація (Custom Authentication)

- Як працює:

- Залежить від конкретного API
- Може включати кілька параметрів або нестандартні заголовки
- Часто комбінує елементи стандартних методів

- Реалізація в n8n:
  - Використовуйте тип облікових даних "Generic"
  - Налаштуйте власні заголовки або параметри
  - Для складної логіки можна використати вузли Function

Приклад: НМАС-автентифікація:

```
// У вузлі Function const crypto = require('crypto'); const timestamp = Math.  
floor(Date.now() / 1000).toString(); const message = timestamp + 'GET' + '/ap  
i/resource'; const signature = crypto .createHmac('sha256', 'your_secret_ke  
y') .update(message) .digest('hex'); return { json: { headers: { 'X-Timestam  
p': timestamp, 'X-Signature': signature } } };
```

---

## Обробка відповідей API та помилок

Правильна обробка відповідей і помилок — ключ до стабільних і надійних робочих процесів.

---

### Обробка успішних відповідей

- Парсинг JSON
  - Більшість API повертають відповіді у форматі JSON
  - n8n автоматично парсить JSON при встановленому параметрі "Response Format = JSON"
  - Отримані дані доступні в наступних вузлах
- Витяг даних
  - Використовуйте вузол Set для витягування необхідних полів
  - Трансформуйте дані у потрібний формат
  - Для вкладених структур використовуйте вирази (expressions)

- **Обробка пагінації**
  - Налаштуйте пагінацію у вузлі HTTP Request
  - Опрацюйте кілька сторінок результатів
  - Об'єднайте дані з різних сторінок

Приклад трансформації відповіді:

```
// Витяг певних полів з відповіді const items = $input.item.json.data.items;  
const simplified = items.map(item => ({ id: item.id, name: item.attributes.name, created: item.attributes.created_at })); return { json: { items: simplified } };
```

## Стратегії обробки помилок

- **Перевірка статус-кодів**
  - Використовуйте вузол IF для перевірки статус-коду
  - Відповідно обробляйте 4xx та 5xx помилки
  - Наприклад: повторювати запити при 5xx, але не при 4xx
- **Парсинг помилок**
  - Витягніть повідомлення про помилку й код із відповіді
  - Запишіть детальну інформацію в журнал
  - Надсилайте змістовні повідомлення про помилку
- **Повторні спроби (Retry logic)**
  - Реалізуйте експоненціальну затримку при повторних спробах
  - Встановіть ліміт кількості спроб
  - Додавайте затримку між запитами

- **Альтернативні варіанти (Fallback)**

- Використовуйте альтернативні джерела даних, якщо основне API недоступне
- Кешуйте попередні успішні відповіді
- Забезпечте "м'яке" зниження функціональності при помилках

---

## Приклад обробки помилок у Function-вузлі після HTTP Request

```
// У вузлі Function після HTTP Request const response = $input.item.json; if
(response.statusCode >= 400) { // Витяг деталей помилки const errorMessage =
response.body.error || 'Невідома помилка'; const errorCode = response.body.co
de || response.statusCode; // Лог помилки console.log(`API Error: ${errorCod
e} - ${errorMessage}`); // Структурована відповідь про помилку return { json:
{ success: false, error: { code: errorCode, message: errorMessage, timestamp:
new Date().toISOString() } } }; } // Успішна відповідь return { json: { succe
ss: true, data: response.body.data } };
```

---

## Розширення можливостей через HTTP Request і концепції API

Завдяки гнучкості вузла HTTP Request і розумінню принципів роботи з API ви можете вийти за межі вбудованих інтеграцій n8n. Це відкриває можливість підключення практично до будь-якого сервісу з API, перетворюючи n8n на універсальну платформу автоматизації.

---

## Credential-Only Nodes та Користувацькі операції

Хоча n8n надає окремі вузли для багатьох популярних сервісів, іноді потрібно працювати з сервісами:

- які не мають вузла в n8n
- або вимагають специфічних операцій, не реалізованих у стандартному вузлі

Для таких випадків n8n підтримує **Credential-Only Nodes** та **кастомні API-запити**.

---

## Credential-Only Nodes

## Що це таке?

- Вузли, які надають лише автентифікацію (OAuth2, API Key тощо)
- Не мають власних дій (немає окремого вузла в палітрі nodes)
- Використовуються разом із HTTP Request вузлом для доступу до API

## Як їх впізнати?

- Зазначені в розділі **Credentials**
- Часто мають суфікс типу (*OAuth2*)
- Відсутні у списку вузлів для побудови процесів

## Переваги:

- Просте налаштування автентифікації
- Коректне виконання OAuth-потоків
- Автоматичне оновлення токенів

---

## Як використовувати Credential-Only вузли

### Приклад використання OAuth2:

1. Створіть облікові дані типу "Service OAuth2 API"
2. Вкажіть параметри (authorization URL, token URL, scope)
3. Пройдіть авторизацію
4. Додайте HTTP Request вузол
5. Оберіть щойно створені облікові дані
6. Налаштуйте endpoint, параметри, заголовки тощо
7. Надішліть запит до API

---

## Поширені сервіси з Credential-Only автентифікацією:

- Сервіси з OAuth2 без вузлів в n8n
  - Спеціалізовані API
  - Прості сервіси з Basic/Auth Token
-

# Розширення наявних вузлів через API-запити

Багато вузлів n8n покривають лише базовий функціонал сервісів. Якщо потрібна нестандартна дія або підтримка нових можливостей — використовуйте кастомні HTTP-запити.

---

## Коли використовувати Custom API Calls

### Операції, яких бракує:

- Немає підтримки окремої функції
- API змінився / додано нову можливість
- Потрібна бета-функція

### Кастомні ендпоінти:

- Внутрішні API
- Приватні чи розширені ендпоінти
- Корпоративні API з нестандартною структурою

### Складні параметри:

- Фільтрація, пагінація, складні запити
  - Масові операції
  - Нетипові схеми
- 

## Як реалізувати

- Використання наявних облікових даних:
  - Створіть базовий вузол (наприклад, Dropbox)
  - Потім додайте HTTP Request вузол
  - Виберіть ті самі облікові дані
  - Надішліть запит до потрібного endpoint

- **Розширене вилучення облікових даних** (просунутий рівень):
    - Використовуйте вузол Function для отримання токена чи ключа
    - Додайте його вручну в заголовок HTTP Request
    - Потрібно для складної автентифікації або нестандартного потоку
- 

## Гібридні робочі процеси (Hybrid Workflows)

- Використовуйте стандартні вузли для типових операцій
  - Додавайте вузли HTTP Request для спеціалізованих дій
  - Обробляйте та об'єднуйте дані з обох підходів
- 

## Приклад: розширення інтеграції Slack

```
// Стандартний вузол Slack — для базових дій // HTTP Request — для розширених можливостей // 1. Налаштуйте HTTP Request: // Метод: POST // URL: https://slack.com/api/conversations.history // Заголовки: Content-Type: application/json // Авторизація: використовуйте Slack OAuth2 облікові дані // 2. Параметри запиту: // channel: C01234ABCDE // limit: 100 // inclusive: true // 3. Обробляйте відповідь у наступних вузлах
```

## Створення власних API-ендпоінтів

n8n також може працювати як **провайдер API**, дозволяючи створювати **власні ендпоінти**, які запускають робочі процеси за запитом.

---

## Webhook-вузол як API-ендпоінт

- **Базове налаштування:**
  - Додайте вузол Webhook першим у вашому робочому процесі
  - Встановіть HTTP-метод (GET, POST тощо)
  - Налаштуйте автентифікацію, якщо потрібно
  - Активуйте робочий процес — згенерується URL webhook'у



- **Обробка запиту:**
    - Доступ до даних запиту через вирази
    - Обробка query-параметрів, заголовків і тіла
    - Додайте валідацію та обробку помилок
  - **Налаштування відповіді:**
    - Встановіть відповідний статус-код
    - Задайте заголовки відповіді
    - Поверніть структуровані дані в тілі відповіді
- 

## Розширені API-шаблони

- **RESTful API для ресурсів**
    - Окремий робочий процес для кожного ресурсу (наприклад, "customers")
    - CRUD-операції (Create, Read, Update, Delete)
    - Шляхові параметри (наприклад, `/customers/:id`)
  - **Версування API**
    - Додавайте версію в шлях: `/v1/api/...`
    - Підтримуйте декілька версій одночасно
    - Документуйте зміни та депрекейти
  - **Документація API**
    - Створюйте документацію для ваших endpoint'ів
    - Додавайте приклади запитів і відповідей
    - Вказуйте вимоги до автентифікації
  - **Безпека**
    - Обов'язково реалізуйте автентифікацію
    - Санітуйте всі вхідні дані
    - Впроваджуйте rate limiting
    - Логуйте доступи й помилки
-

## Приклад: Власне CRUD API для клієнтів

```
GET /api/customers - Webhook (GET) - Function: запит до бази - Set: форматування відповіді
POST /api/customers - Webhook (POST) - Function: валідація - Function: створення запису - Set: відповідь
GET /api/customers/:id - Webhook (GET + path param) - Function: вибір клієнта - Set: відповідь
PUT /api/customers/:id - Webhook (PUT + path param) - Function: оновлення - Set: відповідь
DELETE /api/customers/:id - Webhook (DELETE + path param) - Function: видалення - Set: відповідь
```

Використовуючи **Credential-Only** вузли, кастомні API-запити та власні Webhook API, ви можете подолати обмеження стандартних інтеграцій і побудувати надзвичайно гнучкі автоматизовані рішення в n8n. Це відкриває шлях до роботи практично з будь-яким сервісом або реалізації будь-якої бізнес-логіки.

## 🌟 Розділ 6: Розширені функції та можливості ШІ

Коли ви вже впевнено почуваетесь з базовим функціоналом n8n, саме час перейти до **просунутих можливостей**, які допоможуть створювати ще потужніші, гнучкіші й інтелектуальніші робочі процеси.

### Субпроцеси та спільне використання робочих процесів

Субпроцеси (Subworkflows) дозволяють **модульно структурувати автоматизацію**, розділяючи складні сценарії на **багаторазові компоненти**, покращуючи підтримку та дозволяючи спільну роботу.

## Створення та використання субпроцесів (Subworkflows)

### Що таке субпроцеси?

- Окремі робочі процеси, які можна викликати з інших
- Функціонують як **функції або підпрограми** в програмуванні

- Дозволяють організовувати складні автоматизації в компактні, зрозумілі частини
- 

## Створення субпроцесу

- Створіть робочий процес з тригером та обробкою
  - Збережіть і активуйте його
  - Після цього його можна буде викликати з інших процесів
- 

## Виконання субпроцесу

- Використовуйте вузол "Execute Workflow" у головному процесі
  - Оберіть субпроцес зі списку
  - Налаштуйте передачу та отримання даних між процесами
- 

## Передача даних між процесами

- **Input:** Передайте вхідні дані з головного процесу
  - **Output:** Отримайте результати з субпроцесу
  - Використовуйте **узгоджені структури даних** для стабільної взаємодії
- 

## Приклади застосування

- **Збагачення даних:** Повторно використовуваний субпроцес для доповнення інформації про контакти
  - **Система сповіщень:** Єдиний підхід до відправки повідомлень у різних процесах
  - **Автентифікація:** Централізоване керування токенами для API
  - **Обробка помилок:** Єдиний субпроцес для логування та повідомлень про помилки
- 

## Спільне використання та співпраця

### Експорт робочих процесів

- Завантаження у форматі **JSON**
- Можна поділитися з колегами або на форумі n8n
- Додайте інструкції: облікові дані, налаштування

## Імпорт робочих процесів

- Завантаження файлу JSON
  - Налаштуйте облікові дані
  - Адаптуйте під своє середовище
- 

## Командна робота

- Спільне керування обліковими даними
  - Ролі та доступи (RBAC)
  - Журнал змін, контроль версій
- 

## Спільнота n8n

- Публікуйте робочі процеси на [n8n.io/workflows](https://n8n.io/workflows)
  - Використовуйте чужі шаблони
  - Вивчайте кращі практики автоматизації
- 

## Для бізнес-команд та ентерпрайзу

- Стандартизовані шаблони процесів
  - Управління змінами та погодження
  - Централізовані бібліотеки процесів
- 

## Умовна логіка та гнучке галуження

Більше, ніж простий IF

- Складні умови
  - Комбінуйте кілька перевірок з **AND / OR**
  - Вкладені умови
  - Більше операторів (>, <, contains, startsWith)

## Switch-вузол — альтернатива IF

- Працює як `switch/case` у програмуванні
- Направляє дані в залежності від значення

## Merge-вузол

- **Wait for All**: очікування завершення всіх гілок
- **Merge By Index**: поєднання даних за індексами
- **Merge By Property**: об'єднання на основі ключа
- **Append**: просте об'єднання всіх елементів

---

## Приклад: складна маршрутизація

```
const item = $input.item.json; let category; if (item.value > 1000 && item.priority === 'high') { category = 'urgent-high-value'; } else if (item.value > 1000) { category = 'high-value'; } else if (item.priority === 'high') { category = 'urgent'; } else if (item.status === 'new' && item.source === 'referral') { category = 'new-referral'; } else { category = 'standard'; } return { json: { ...item, category } };
```

---

## Webhooks та обробка в реальному часі

### Просунуті шаблони Webhook'ів

### Кастомізація відповіді

- Задавайте власні HTTP статус-коди
- Додавайте кастомні заголовки

- Формуйте структуровану відповідь

## Методи автентифікації

- Basic Auth
- API Key
- JWT
- Перевірка підпису

## Верифікація Webhook

- Перевірка джерела за shared secret
- Підпис + timestamp — для запобігання атак повтору (replay attack)

---

## Приклад: Валідація підпису Webhook

```
// У вузлі Function після Webhook const crypto = require('crypto'); // Отримання деталей запиту const payload = $input.item.json.body; const signature = $input.item.json.headers['x-webhook-signature']; const timestamp = $input.item.json.headers['x-webhook-timestamp']; const secret = 'your_webhook_secret'; // Перевірка, що підпис актуальний (не старший за 5 хвилин) const now = Math.floor(Date.now() / 1000); if (now - parseInt(timestamp) > 300) { return { json: { valid: false, error: 'Webhook timestamp too old' } } }; // Обчислення очікуваного підпису const expectedSignature = crypto.createHmac('sha256', secret).update(`${timestamp}.${JSON.stringify(payload)}`).digest('hex'); // Порівняння підписів if (signature !== expectedSignature) { return { json: { valid: false, error: 'Invalid signature' } } }; // Якщо підпис правильний return { json: { valid: true, data: payload } };
```

---

## Обробка даних у реальному часі

### Стрімінг подій (Event Streaming)

- Обробляйте події по мірі надходження
- Зберігайте стан між подіями

- Реалізуйте агрегування за вікнами часу (windowing)

## Інтеграція з WebSocket

- Підключення до WebSocket API
- Підтримка постійного з'єднання
- Обробка стрімінгових даних у реальному часі

## SSE — Server-Sent Events

- Підписка на потоки подій з сервера
- Обробка подій по мірі надходження
- Контроль стану з'єднання

---

## Приклад: реалізація WebSocket-клієнта

```
// У вузлі Function const WebSocket = require('ws'); const url = $input.item.json.websocketUrl; let messages = []; return new Promise((resolve, reject) => { const ws = new WebSocket(url); ws.on('open', () => { console.log('Connected to WebSocket'); ws.send(JSON.stringify({ type: 'auth', token: $input.item.json.authToken })); }); ws.on('message', (data) => { try { const message = JSON.parse(data); messages.push(message); console.log(`Received message: ${message.type}`); if (messages.length >= 10) { ws.close(); resolve({ json: { messages } }); } } catch (error) { console.error('Error processing message:', error); } }); ws.on('error', (error) => { reject(error); }); setTimeout(() => { ws.close(); resolve({ json: { messages } }); }, 30000); });
```

---

## Реалізація callback-систем

### Webhook Callback Pattern

- Генеруйте унікальні callback-URL
- Чекайте запитів від зовнішніх систем
- Продовжуйте виконання процесу після callback'у

## Довгі операції (Long-Running Operations)

- Запускайте асинхронні дії
- Перевіряйте статус через polling або очікуйте callback
- Обробляйте результати при надходженні

## Використання вузла Wait

- Призупиняє виконання
  - Відновлюється після виклику webhook
  - Передає дані між фазами виконання
- 

## Приклад: callback після оплати

Сценарій:

1. **Webhook Node** — прийом запиту на оплату
  2. **Function Node** — генерація унікального ID платежу
  3. **HTTP Request** — ініціація платежу через API провайдера
  4. **Wait Node** — очікування callback за унікальним URL
  5. **Function Node** — перевірка завершення платежу
  6. **Send Email** — підтвердження успішної оплати
- 

## Висновок:

Опановуючи ці **просунуті можливості n8n**, ви зможете:

- будувати **інтелектуальні й гнучкі автоматизації**,
  - працювати з **великими обсягами даних у реальному часі**,
  - реалізовувати **callback-архітектуру**,
  - забезпечувати **високу надійність і масштабованість**.
- 

## Інтеграція ШІ в n8n

n8n надає потужні можливості для інтеграції штучного інтелекту у ваші робочі процеси. Цей розділ розкриває, як ефективно використовувати AI-сервіси та створювати інтелектуальні автоматизації.



---

## Підключення до ШІ-сервісів

n8n дозволяє легко підключатися до провайдерів штучного інтелекту, навіть без глибоких знань у сфері машинного навчання.

---

## Підтримувані платформи штучного інтелекту

- **OpenAI**
  - Доступ до моделей GPT для генерації та доповнення текстів
  - Генерація зображень через DALL-E
  - Використання embedding для семантичного пошуку
  - Тонке налаштування моделей під власні задачі
- **Google AI**
  - Мовні моделі Google
  - Комп'ютерний зір (Vision AI)
  - Сервіси перекладу
  - Перетворення мови в текст і навпаки
- **Hugging Face**
  - Тисячі open-source моделей
  - Спеціалізовані моделі для окремих завдань
  - Підтримка кастомних моделей
  - Експерименти з новітніми дослідженнями в AI
- **Anthropic**
  - Робота з моделями Claude
  - Реалізація чат-ботів і віртуальних асистентів
  - Аналіз і генерація текстів
  - Глибока обробка змісту